République Démocratique du Congo
Ministère de l'Enseignement Supérieur et Universitaire

Université de Kinshasa

Faculté des Sciences et Technologies
B.P. 190 Kinshasa XI



Contrôle décentralisé d'un essaim de robots par réseaux de neurones sur graphe

---

Mémoire présenté et défendu en vue de l'obtention
du grade de **Master en Sciences et Technologies**

Filière : Informatique
Option : Intelligence artificielle et Sciences des données

Présenté par

MWANAMPUTU LABEYA Laurent

Sous la direction du

Prof. Dr Selain KASEREKA KABUNGA

Année académique 2025–2026

Épigraphe

Confie-toi en l'Éternel de tout ton cœur, et ne t'appuie pas sur ta sagesse ; reconnais-le dans toutes tes voies, et il aplanira tes sentiers.

Proverbes 3 :5-6

Dédicace

À mes parents, qui ont tant donné pour que je puisse étudier et qui n'ont jamais cessé de croire en moi.

À mes frères et sœurs, dont les encouragements m'ont porté quand le chemin devenait difficile.

Et à toutes celles et tous ceux qui, de près ou de loin, ont compté dans l'aboutissement de ce travail.

Je vous dédie ce mémoire.

Laurent MWANAMPUTU LABEYA

Remerciements

Au terme de notre Master en Informatique, option Intelligence artificielle et Sciences des données, à l'Université de Kinshasa (UNIKIN), nous exprimons notre gratitude à toutes les personnes qui ont contribué à notre formation et à la réalisation de ce travail.

Avant tout, nous rendons grâce à notre Seigneur et Sauveur Jésus-Christ pour sa grâce, sa sagesse et la force qu'il nous a accordées tout au long de notre parcours. À lui soient la gloire et l'honneur.

Nous remercions les autorités académiques de l'UNIKIN, ainsi que les professeurs, chefs de travaux et assistants de la Faculté des Sciences et Technologies, pour leurs enseignements et leur encadrement.

Nos sincères remerciements vont au Professeur Docteur **Selain KASEREKA KABUNGA**, qui a accepté de diriger ce travail. Nous lui sommes reconnaissants pour ses conseils et ses orientations.

Nous exprimons toute notre reconnaissance à nos chers parents, **MWANAMPUTU ONGOS Thierry** et **NDINGAMBOTE NZOLAMESO Jeannette**, pour leur éducation, leurs prières, leurs sacrifices et leurs encouragements. Que le Seigneur leur accorde santé et longue vie.

Nous remercions également notre frère, **Paradi MWANAMPUTU**, nos sœurs, **MWANAMPUTU AWIKIES Donnel** et **MWANAMPUTU Georgina**, ainsi que toute la famille **MWANAMPUTU**, nos cousins et cousines, pour leur affection et leur soutien.

Enfin, nous remercions nos collègues et amis pour les encouragements et les moments partagés durant notre parcours universitaire.

À toutes les personnes qui nous ont soutenus de près ou de loin, nous adressons nos sincères remerciements.

MWANAMPUTU LABEYA Laurent

Liste des abréviations

CPU	Central Processing Unit (processeur central)
CSV	Comma-Separated Values (tableau de valeurs séparées)
DPI	Dots Per Inch (points par pouce)
GAT	Graph Attention Network (réseau à attention sur graphe)
GNN	Graph Neural Network (réseau de neurones sur graphe)
GPU	Graphics Processing Unit (processeur graphique)
IA	Intelligence Artificielle (domaine de l'étude)
JSON	JavaScript Object Notation (format de données structurées)
LiDAR	Light Detection And Ranging (mesure de distances par lumière)
MLP	Multi-Layer Perceptron (perceptron multicouche)
MPNN	Message Passing Neural Network (réseau à passage de messages)
MSE	Mean Squared Error (erreur quadratique moyenne)
PD	Proportional-Derivative (commande proportionnelle et dérivée)
PDF	Portable Document Format (format de document)
PNG	Portable Network Graphics (format d'image matricielle)
QP	Quadratic Programming (optimisation quadratique)
ReLU	Rectified Linear Unit (fonction d'activation)
SHA-256	Secure Hash Algorithm, 256 bits (empreinte d'un fichier)
SiLU	Sigmoid Linear Unit (fonction d'activation)
SVG	Scalable Vector Graphics (format de dessin vectoriel)

Table des figures

1.1 Graphe extrait d'un départ enregistré à 24 robots. Les traits relient les voisins directs. La portée est de 3 m. Données issues de la trajectoire du rectangle, graine 8500, campagne du modèle structuré. 5

3.1 Exemples de tracés du catalogue 21
3.2 Courbes extraites des journaux d'entraînement. En haut, l'imitation du modèle initial est mesurée par une MSE. En bas, le modèle structuré v2 utilise un coût composite. Les deux ordonnées n'ont pas la même signification. 30

4.1 Lecture d'un cas réservé de l'ellipse 39
4.2 Contacts avant et après correction 40
4.3 Quatre états d'une trajectoire enregistrée du rectangle à 24 robots, graine 8500, GAT structuré brut. Les cercles vides indiquent les positions finales enregistrées, les points pleins les robots à l'instant indiqué. Le cercle initial est une étape du protocole. 43
4.4 Temps de stabilisation décomposé à partir du protocole et des moyennes mesurées du GAT structuré. L'allocation domine à 64 robots. La dernière partie couvre le temps entre la libération du mouvement et la stabilisation. 44
4.5 Erreur et vitesse après libération 45
4.6 Trajectoires enregistrées pendant le transport du carré avec dix obstacles, graine 8901, GAT initial protégé. Les disques noirs représentent les obstacles à leur rayon physique. Les traits continus suivent les centres des robots ; les cercles vides indiquent leur état final. Le trait discontinu suit le centre de référence. La préparation n'est pas tracée. 48

Liste des tableaux

2.1 Analyse du travail connexe 1 15
2.2 Analyse du travail connexe 2 15
2.3 Analyse du travail connexe 3 16
2.4 Choix du système et portée des validations. 17

3.1 Paramètres communs aux principales campagnes décrites. Les dimensions de l'espace et de la forme varient selon l'effectif. 19
3.2 Dimensions des blocs des deux modèles. 24
3.3 Partitions de la collecte élargie du modèle initial. 26
3.4 Contenu d'un exemple local avant prétraitement. 26
3.5 Profils contrôlés des campagnes multi-effectifs. 33
3.6 Organisation des composants de la version actuelle. 34

4.1 Principales campagnes discutées. « Cas » désigne une combinaison de forme, de départ et d'effectif ou de scène ; il ne signifie pas toujours un départ distinct. 37
4.2 Comparaison réservée du GAT initial 38
4.3 Cas de l'ellipse, douze robots, graine 7300. 38
4.4 Pilote multi-effectifs avant le modèle structuré. Données issues du rapport 012. 41
4.5 Test réservé du GAT structuré v2. Les erreurs et temps sont des moyennes sur six cas par effectif. Les temps incluent toutes les phases de préparation. Données issues du rapport 014. 42
4.6 Détail des cas réservés du modèle structuré v2. 46
4.7 Campagne finale avec obstacles proches des cibles. Les deux dernières colonnes sont les maxima entre les cas, et non des moyennes. La déformation est mesurée pendant le transport. Données issues du rapport 019. 47

Résumé

Ce mémoire étudie la formation et le déplacement d'un essaim de robots dans un simulateur en deux dimensions. Chaque robot calcule une accélération à partir de son état, de sa cible et de ses voisins directs. Les places sont attribuées par un protocole classique d'échanges locaux ; leur attribution est distincte de la commande apprise. Le catalogue comprend 56 noms de lettres, de nombres et de figures, correspondant à 54 géométries différentes.

Les travaux liés abordent des tâches et des hypothèses différentes. Leurs résultats ne suffisent pas à établir l'apport d'une commande apprise dans notre propre protocole. Il faut aussi distinguer le rôle du réseau de celui de l'attribution et de la protection classiques.

Les données d'entraînement sont produites par un contrôleur de référence dans le simulateur. L'apprentissage supervisé consiste à reproduire ses accélérations. Un premier réseau à attention sur graphe, entraîné à douze robots, réussit les neuf cas réservés lorsqu'il est protégé par un filtre classique. Sans ce filtre pendant le mouvement, il échoue dans ces neuf cas et provoque 54 contacts. Une collecte de correction réduit les contacts d'une campagne ultérieure sans rendre le mouvement brut suffisamment fiable.

Une seconde architecture associe un mouvement explicite vers la cible et des corrections apprises à partir des voisins. Elle réussit 18 cas réservés à 12, 24 et 64 robots sans contact ni filtre pendant les phases de mouvement apprises. Ces essais couvrent trois formes et deux départs par effectif, près d'un cercle de préparation. La navigation avec obstacles est évaluée séparément. Elle repose sur un trajet classique calculé sur une carte connue et sur une commande protégée. Les huit cas de la campagne finale réussissent pour chacun des deux contrôleurs comparés.

Les résultats montrent l'intérêt d'une architecture structurée dans les conditions étudiées. Ils ne permettent pas de conclure à une réussite sur tous les départs, toutes les formes ou des robots réels. Le mémoire distingue les erreurs d'imitation, les mesures de trajectoire et les apports des composants classiques pour préciser ce que les résultats permettent de conclure.

Mots-clés : robotique en essaim, réseaux sur graphes, imitation supervisée, formation, simulation, évaluation.

Abstract

This thesis studies swarm formation and motion in a two-dimensional simulator. Each robot computes an acceleration from its own state, its assigned target and its direct neighbours. Target allocation follows a separate classical protocol based on local exchanges. The catalogue contains 56 names for letters, numbers and shapes, representing 54 distinct geometries.

Related studies use different tasks and assumptions. Their results do not establish the contribution of a learned controller within our own protocol. Target allocation and classical protection must also be distinguished from the learned policy.

Training examples are generated by a reference controller in simulation. Supervised learning is used to imitate its accelerations. A first graph attention model, trained with twelve robots, succeeds in nine reserved cases when protected by a classical filter. Without that filter during motion, it fails in all nine cases and produces 54 contact events. Adding corrective examples reduces contacts in a later campaign, but the unfiltered policy remains unreliable.

A second architecture combines an explicit motion towards the target with learned corrections from neighbours. It succeeds in 18 reserved cases with 12, 24 and 64 robots, without contact or a safety filter during learned motion phases. These cases cover three shapes and two initial configurations per swarm size, close to a preparation ring. Obstacle navigation is evaluated separately and uses classical planning on a known map together with protected motion control. All eight cases in the final navigation campaign succeed for both compared controllers.

These results support the structured architecture within the evaluated conditions. They do not establish general success across arbitrary initial states, all catalogue shapes or physical robots. The thesis separates imitation error, trajectory metrics and the contributions of classical components to make the scope of its conclusions clear.

Keywords : swarm robotics, graph neural networks, supervised imitation, formation, simulation, evaluation.

Table des matières

Épigraphe **i**

Dédicace **ii**

Remerciements **iii**

Liste des figures **v**

Liste des tableaux **vi**

Résumé **vii**

Abstract **viii**

Introduction générale **1**

1 Contexte de l'étude 1
2 Problématique 1
3 Hypothèses 1
4 Objectifs poursuivis 1
5 Méthodes et techniques utilisées 2
6 Contribution de l'étude 2
7 Plan sommaire 2

1 Concepts de base **3**

1.1 Introduction 3
1.2 Essaim, localité, décentralisation 3
1.2.1 Essaim 3
1.2.2 Localité 3
1.2.3 Décentralisation 3
1.3 Voisins, portée de communication et graphe de proximité 4
1.3.1 Voisins 4
1.3.2 Portée de communication 4
1.3.3 Graphe de proximité 5
1.4 Réseaux de neurones sur graphes 5
1.4.1 Place de l'apprentissage dans le système 6
1.4.2 Calcul interne et passage de messages 6
1.4.3 Différence entre communication et décision 7
1.4.4 Un même réseau pour plusieurs robots 7

1.5 Évaluation de la formation . . . 7
1.6 Règle écrite et politique apprise . . . 8
1.6.1 Organisation des données . . . 9
1.6.2 Rôle de l'ensemble de validation . . . 9
1.6.3 Missions du groupe . . . 9
1.7 Conclusion partielle . . . 10

2 État de l'art **11**
2.1 Introduction . . . 11
2.2 Méthodes traditionnelles . . . 11
2.2.1 Règles locales et consensus distribué . . . 11
2.2.2 Formation de formes par contrôle décentralisé . . . 12
2.2.3 Conception explicite de la loi de contrôle . . . 12
2.3 Méthodes avancées . . . 13
2.3.1 Variantes des GNN et justification de l'agrégation . . . 13
2.3.2 Apprentissage de contrôleurs décentralisés par GNN . . . 13
2.3.3 Méthodes d'apprentissage . . . 14
2.3.4 De la règle écrite à la règle apprise . . . 14
2.4 Quelques travaux connexes . . . 14
2.4.1 Decentralized controllers for shape generation with robotic swarms . 15
2.4.2 Learning Decentralized Controllers for Robot Swarms with Graph Neural Networks . . . 15
2.4.3 End-to-end decentralized formation control using a graph neural network-based learning method . . . 16
2.5 Positionnement . . . 16
2.5.1 Lecture critique des comparaisons . . . 16
2.5.2 Informations disponibles . . . 16
2.5.3 Portée de la contribution . . . 17
2.6 Conclusion partielle . . . 17

3 Méthodologie et solution proposée **18**
3.1 Introduction . . . 18
3.2 Organisation du système . . . 18
3.3 Modèle physique . . . 18
3.3.1 Pourquoi commander une accélération ? . . . 19
3.4 Représentation des formes . . . 19
3.5 Préparation et attribution des places . . . 20
3.5.1 Pourquoi une étape circulaire ? . . . 20
3.5.2 Réservation locale et conflits . . . 22
3.5.3 Exemple de résolution d'un conflit . . . 22

3.6 Contrôleur de référence et protection classique . . . 23
3.7 Premier modèle à attention sur graphe . . . 23
3.7.1 Entrées et calcul de la commande . . . 23
3.7.2 Du fichier d'observation au calcul local . . . 24
3.7.3 Lecture des dimensions de l'architecture . . . 24
3.7.4 Modes brut et protégé . . . 25
3.8 Données et entraînement du modèle initial . . . 25
3.8.1 Origine et séparation des données . . . 25
3.8.2 Contenu d'une observation enregistrée . . . 26
3.8.3 Optimisation et sélection . . . 27
3.8.4 Déroulement d'un entraînement . . . 27
3.9 Corrections et modèle structuré . . . 28
3.9.1 Collecter sur les états rencontrés par le réseau . . . 28
3.9.2 Mouvement nominal et corrections vectorielles . . . 28
3.9.3 Préparation des exemples et coût d'apprentissage . . . 29
3.10 Navigation avec une carte connue . . . 29
3.11 Critères d'évaluation . . . 31
3.11.1 Erreur de formation et maintien . . . 31
3.11.2 Contacts et réussite . . . 31
3.12 Reproductibilité et lecture des résultats . . . 32
3.12.1 Calcul et environnement enregistré . . . 32
3.12.2 Formats des fichiers et provenance . . . 33
3.13 Configurations actuelles et progression de l'apprentissage . . . 33
3.14 Organisation logicielle et niveau de preuve . . . 34
3.15 Conclusion partielle . . . 34

4 Implémentation, expérimentations et résultats 35
4.1 Introduction . . . 35
4.2 Ce que fait l'application . . . 35
4.2.1 Conditions d'une démonstration reproductible . . . 36
4.3 Protocole des campagnes . . . 36
4.3.1 Même départ, même critère . . . 36
4.4 Ce que montre l'entraînement . . . 37
4.5 Limites du premier modèle d'imitation . . . 37
4.5.1 Un cas qui explique le rôle du filtre . . . 38
4.6 Résultats de la collecte de correction . . . 40
4.7 Premier essai avec plusieurs effectifs . . . 41
4.8 Formation à plusieurs effectifs avec le modèle structuré . . . 41
4.8.1 Validation avant le test réservé . . . 41

4.8.2 Résultats du test réservé 42
4.8.3 Pourquoi le temps augmente-t-il autant ? 42
4.8.4 Lecture des dix-huit cas 44
4.9 Transitions entre formes 46
4.10 Navigation et obstacles proches des cibles 46
4.10.1 Difficulté et correction du trajet 46
4.10.2 Campagne finale 47
4.11 Analyse des métriques et des résultats négatifs 49
4.12 Lecture d'ensemble 49
4.13 Conclusion partielle 49

Discussion **50**
Introduction 50
Interprétation des résultats 50
Comparaison avec les travaux antérieurs 51
Menaces à la validité 52
Validité interne 52
Validité externe 52
Validité de construit 53
Implications pour la conception de contrôleurs décentralisés 53
Considérations pratiques et sociales 54
Suite expérimentale 54
Conclusion partielle 55

Conclusion générale et perspectives **56**

Bibliographie **58**

Annexe A — Mode d'emploi **59**

Annexe B — Les expériences **60**

Annexe C — Résultats des simulations **61**

Introduction générale

1 Contexte de l'étude

La robotique en essaim étudie la coordination de robots par des décisions locales [2]. Nous l'appliquons à la formation de lettres, de nombres et de figures, puis au déplacement de certaines formations autour d'obstacles connus.

Le réseau apprend une accélération à partir des exemples d'un contrôleur de référence programmé. L'attribution des places et la préparation du trajet restent classiques.

2 Problématique

Une bonne forme finale peut cacher des contacts pendant le trajet. Une faible erreur d'entraînement ne garantit donc pas une mission réussie. Le filtre classique peut aussi corriger les commandes du réseau.

Nous cherchons à savoir dans quelles conditions une commande locale apprise par imitation permet de former un essaim sans contact. Il faut aussi déterminer le rôle des composants classiques.

3 Hypothèses

Trois hypothèses orientent les essais.

1. Un réseau partagé peut apprendre une commande de formation à partir de sa cible et de ses voisins.
2. Une commande qui conserve explicitement l'attraction et le freinage peut faciliter cet apprentissage.
3. Un même modèle peut être utilisé avec plusieurs effectifs, mais sa réussite doit être vérifiée pour chacun.

La simulation utilise un espace borné, des observations exactes, des échanges sans perte et des obstacles connus. Les essais ne couvrent pas les capteurs bruités, les pannes ou les robots réels.

4 Objectifs poursuivis

L'objectif général est de construire et d'évaluer une commande locale de formation. Cinq objectifs guident ce travail.

— simuler des déplacements à accélérations bornées ;
— construire les places des formes et régler leur attribution ;
— produire les données du contrôleur de référence et entraîner les modèles ;
— comparer les trajectoires selon la formation, la stabilisation et les contacts ;
— examiner séparément le passage à plusieurs effectifs et la navigation.

Les lettres et les nombres sont des consignes à former, et non des images que le réseau doit reconnaître. La réussite demande une forme maintenue, des vitesses faibles et aucun contact pendant l'épisode.

5 Méthodes et techniques utilisées

L'apprentissage supervisé utilise les observations et les accélérations du contrôleur de référence. Entraînement, validation et test sont séparés ; les poids ne changent plus lors du test réservé.

Les comparaisons utilisent les mêmes départs. Les trajectoires sont archivées pour vérifier les commandes et recalculer les métriques.

6 Contribution de l'étude

Le travail réunit simulation, attribution, apprentissage et démonstration web, dont les résultats peuvent être vérifiés.

Le modèle structuré réussit dix-huit cas réservés à trois effectifs, avec des départs contrôlés. La navigation hybride réussit huit cas par contrôleur. La méthode et les résultats précisent les limites de ces campagnes.

7 Plan sommaire

Hormis l'introduction générale, la discussion et la conclusion générale, ce mémoire comprend quatre chapitres.

1. **Concepts de base**. Ce chapitre présente les robots, les graphes et l'apprentissage.
2. **État de l'art**. Il compare les méthodes existantes et situe l'étude.
3. **Méthodologie et solution proposée**. Il décrit les modèles, les données et les critères d'évaluation.
4. **Implémentation, expérimentations et résultats**. Il présente le programme et les essais.

Chapitre 1

Concepts de base

1.1 Introduction

Ce chapitre présente les notions utilisées dans le mémoire. Il part du robot individuel, puis décrit les échanges de voisinage et le principe de l'apprentissage à partir d'exemples.

1.2 Essaim, localité, décentralisation

1.2.1 Essaim

Un essaim est un groupe de robots qui coordonnent leurs actions par des interactions locales. Des règles de séparation, d'alignement et de regroupement peuvent produire un déplacement collectif [6]. Former une figure demande en plus de répartir les robots sur des places précises.

Dans notre simulateur, chaque robot possède une position et une vitesse. Le contrôleur demande une accélération, puis le moteur de simulation calcule le déplacement. Ce modèle représente une inertie simple, sans représenter en détail les roues, la batterie ou les capteurs réels.

1.2.2 Localité

Une décision est locale lorsqu'elle utilise les observations propres du robot et les informations de ses voisins directs. La portée détermine quels robots sont voisins, sans donner accès à tout l'essaim.

1.2.3 Décentralisation

La décentralisation concerne la répartition des décisions entre robots. Les robots peuvent néanmoins disposer d'informations communes. Les robots peuvent partager une mission tout en calculant séparément leurs commandes.

Environnement et informations disponibles

Dans ce mémoire, l'environnement est fermé, car le programme en définit les limites et les conditions. Les dimensions de l'espace, les limites physiques, les cibles et les obstacles sont connus au lancement. Aucun événement extérieur imprévu ne vient modifier la scène

pendant les expériences rapportées. Cette description ne signifie pas que chaque robot reçoit toutes les positions. Le simulateur connaît l'état complet, mais ne transmet au contrôleur que les informations autorisées.

Le robot observe sa position, sa vitesse, sa cible et les déplacements relatifs de ses voisins directs. La portée de communication utilisée dans les principales campagnes est de trois mètres. Les mesures sont exactes dans le modèle simulé. Il n'y a ni bruit de mesure, ni retard variable, ni perte aléatoire de messages. La carte d'obstacles utilisée pour préparer les détours est une information commune donnée au planificateur classique.

Le caractère décentralisé doit donc être précisé. La décision locale de mouvement n'utilise pas l'état physique de robots éloignés. L'attribution des places s'appuie sur des échanges entre voisins. En revanche, la définition de la mission et la planification d'un trajet sur une carte connue restent communes au groupe. Dire que toute la chaîne serait entièrement décentralisée masquerait cette différence.

1.3 Voisins, portée de communication et graphe de proximité

1.3.1 Voisins

Les voisins sont les robots dont les états relatifs sont accessibles à une décision. Cette liste peut changer au cours du mouvement.

Représentation des positions

Chaque position contient deux coordonnées. Dans une matrice, une ligne correspond à un robot et les deux colonnes indiquent ses coordonnées dans le plan. Pour conserver ces nombres dans un seul vecteur, on peut lire les lignes dans l'ordre.

Prenons deux robots placés aux coordonnées (1, 2) et (3, 4). Le tableau de deux lignes devient la liste (1, 2, 3, 4). Cette opération change le rangement des nombres, pas les positions. Le réseau ne reçoit pas cette liste globale de tout l'essaim ; il utilise l'état propre du robot et ses voisins.

Les cibles sont rangées dans un tableau comparable. La géométrie fixe les places disponibles. L'attribution détermine ensuite celle de chaque robot. Permuter les identifiants des robots sans déplacer leurs positions ne change donc pas la forme visible.

1.3.2 Portée de communication

La portée est la distance maximale autorisant un échange direct. Un message peut circuler plus loin par plusieurs échanges, si le protocole le prévoit. Cela ne donne pas au réseau de mouvement un accès immédiat aux états éloignés.

1.3.3 Graphe de proximité

Le graphe représente les échanges possibles. Un sommet correspond à un robot ; une arête relie deux robots lorsque leur distance ne dépasse pas la portée.

Le graphe évolue avec les déplacements des robots. Sa connexité signifie qu'il existe un chemin entre chaque paire de robots ; elle ne signifie pas que tous communiquent directement.

La Figure 1.1 présente un graphe extrait du départ enregistré à 24 robots.

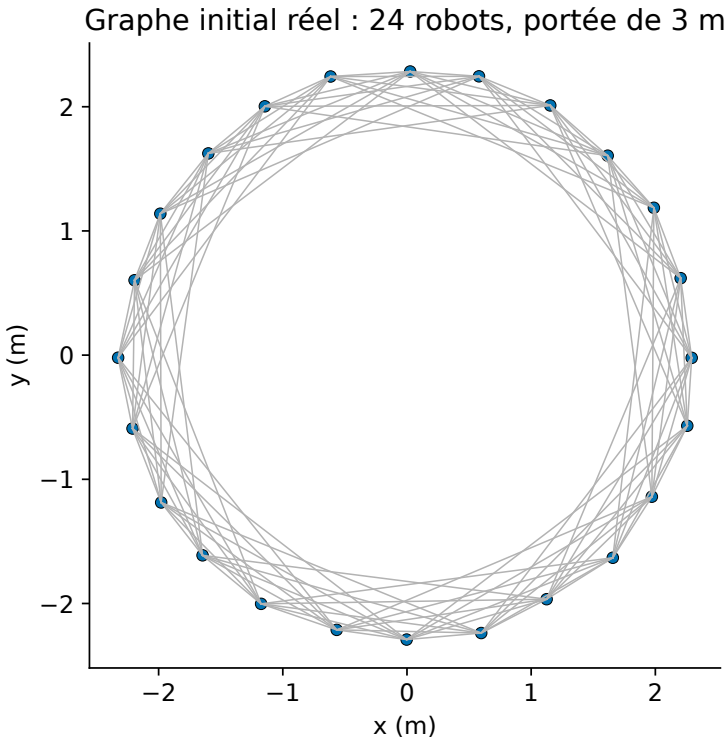


FIGURE 1.1 – Graphe extrait d'un départ enregistré à 24 robots. Les traits relient les voisins directs. La portée est de 3 m. Données issues de la trajectoire du rectangle, graine 8500, campagne du modèle structuré.

Les messages d'attribution peuvent se propager sur plusieurs échanges successifs. Le réseau de mouvement étudié, lui, effectue une seule agrégation des voisins directs à chaque décision. Il faut distinguer cette opération de la propagation répétée des réservations.

1.4 Réseaux de neurones sur graphes

Une couche linéaire calcule une expression $Wx+b$. Des fonctions non linéaires complètent ces transformations. Le nombre de couches linéaires, leur largeur et le nombre d'échanges sur le graphe sont trois caractéristiques différentes. Une largeur de 64 signifie que certains calculs utilisent 64 composantes ; elle ne signifie pas que le modèle contient 64 robots ou 64 couches.

Un réseau sur graphe construit une représentation à partir des voisins. Une somme ou une moyenne traite leurs contributions de façon uniforme. L'attention attribue des poids qui dépendent des observations [10]. Le réseau calcule un score pour chaque voisin, puis transforme ces scores en poids dont la somme vaut un. Il combine les contributions selon ces poids. Lorsqu'il n'y a aucun voisin, la contribution reçue est nulle. Le partage des paramètres permet de traiter plusieurs tailles de groupe, mais les trajectoires doivent encore être vérifiées pour chacune.

1.4.1 Place de l'apprentissage dans le système

L'intelligence artificielle (IA) regroupe différentes façons de construire une décision à partir de données ou de règles. Ici, nous utilisons un apprentissage supervisé. Le réseau reçoit des exemples d'observations et de commandes, puis ses paramètres sont ajustés. Il ne connaît pas le résultat attendu parce qu'on lui aurait donné le nom de la forme comme une classe à reconnaître.

La conception du réseau et son apprentissage sont deux étapes différentes. Nous choisissons les informations accessibles, les fonctions d'activation et l'organisation du réseau. L'optimisation ajuste ensuite les poids de cette architecture. La préparation circulaire, les limites physiques et les critères de réussite ne sont pas appris de cette manière ; ils restent définis dans le programme.

1.4.2 Calcul interne et passage de messages

Un perceptron multicouche, ou MLP pour *Multi-Layer Perceptron*, transforme un vecteur de taille fixée. Ses couches combinent les valeurs d'entrée, puis les activations introduisent des réponses non linéaires. Dans nos modèles, de petits MLP encodent l'état propre et les informations d'un voisin. Cela décrit leur calcul interne, pas un autre contrôleur entraîné à côté du GAT.

Un réseau à passage de messages, ou MPNN pour *Message Passing Neural Network*, organise en plus les interactions entre éléments [3]. Les messages sont calculés puis regroupés pour produire une représentation locale. Le GAT ajoute une pondération par attention. Les mêmes fonctions sont réutilisées pour les voisins présents, dont le nombre peut varier.

Cette distinction répond à une question fréquente. Huit transformations linéaires ne signifient pas huit échanges de voisinage. Une partie de ces transformations prépare les caractéristiques ; une autre calcule les scores ou la sortie. Dans notre commande, l'agrégation de voisins est réalisée une fois à chaque décision. Les échanges répétés des réservations appartiennent au protocole d'attribution.

La largeur interne ne donne pas non plus la taille de l'essaim. Un encodage de 64 composantes représente une observation avec 64 nombres. Il n'exige pas 64 robots. À l'inverse, un modèle de largeur 48 peut être exécuté sur un groupe de 64 robots si la

structure des entrées le permet. La réussite du mouvement reste à mesurer dans cette configuration.

1.4.3 Différence entre communication et décision

Recevoir une information n'oblige pas à suivre le voisin. Un robot peut savoir qu'un autre s'approche et décider de freiner, de dévier ou de conserver sa direction. Le message décrit une situation ; le contrôleur transforme cette situation en action. Une même information peut donc conduire à des commandes différentes selon la cible et la vitesse propres du robot.

Le graphe joue deux rôles dans notre système. Pendant la réservation, il sert à transmettre des annonces de places. Pendant le mouvement, il détermine quels états relatifs peuvent influencer la commande locale. Les contenus échangés ne sont pas les mêmes. Une annonce propagée par plusieurs robots ne donne pas au réseau un accès direct à toutes leurs vitesses.

La connexité facilite la circulation des réservations, mais elle ne résout pas les conflits à elle seule. Il faut encore une règle commune pour décider quelle demande garde une place. Inversement, un robot momentanément sans voisin peut continuer à calculer une attraction vers sa cible. Cela ne suffit pas à garantir que le groupe entier terminera la mission.

1.4.4 Un même réseau pour plusieurs robots

Partager les paramètres signifie que les mêmes matrices sont utilisées pour tous les robots. Le numéro du robot ne désigne pas un réseau particulier. Deux robots qui recevraient exactement les mêmes caractéristiques de mouvement produiraient la même réponse. Leurs commandes diffèrent selon leurs cibles, leurs vitesses et les voisins présents.

Ce partage réduit le nombre de modèles à entraîner. Il permet aussi d'appliquer le calcul à un ensemble de voisins plus grand ou plus petit. Mais le modèle peut rencontrer une densité ou des vitesses relatives peu représentées dans les exemples. L'architecture peut traiter ces entrées. Il faut ensuite des essais pour savoir si le modèle réussit dans ces conditions.

1.5 Évaluation de la formation

Il faut vérifier que les robots sont près de la forme et qu'ils en couvrent toutes les parties. Si tous se regroupent sur un seul coin du carré, la première réponse peut sembler favorable pour certains points alors que le carré reste vide ailleurs. Une mesure de couverture doit regarder dans les deux sens, des robots vers les places et des places vers les robots.

La forme et son attribution ne sont pas identiques. Permuter deux identifiants tout en gardant les positions ne change pas la figure visible. Pendant le trajet, en revanche, savoir à qui appartient chaque place reste utile pour éviter des demandes concurrentes. Nous utilisons donc une mesure géométrique pour la couverture et un état d'attribution pour vérifier l'organisation.

Une troisième question concerne le temps. Traverser brièvement la bonne configuration ne signifie pas y rester. Les robots peuvent passer au voisinage des cibles avec une vitesse trop élevée et repartir aussitôt. La stabilisation demande à la fois une forme correcte, une faible vitesse et un maintien. Les seuils chiffrés appartiennent au protocole décrit au chapitre de méthode.

Enfin, une image finale ne dit pas si un contact a eu lieu plus tôt. La réussite est une propriété de l'épisode complet. C'est pourquoi l'enregistrement conserve les positions successives et le bilan des contacts, même lorsque la dernière image paraît satisfaisante.

1.6 Règle écrite et politique apprise

L'apprentissage supervisé associe une entrée à une réponse attendue. La classification prédit une catégorie, par exemple un type d'objet. La régression prédit des nombres. Ici, la sortie contient deux composantes d'accélération. Il s'agit donc d'une régression.

Les exemples proviennent d'une règle de commande programmée, appelée contrôleur de référence. À chaque état collecté, elle fournit l'accélération attendue que le réseau apprend à reproduire. Les données sont dites synthétiques parce qu'elles sont produites dans le simulateur. Elles comprennent les observations, les voisinages, les commandes et leur provenance. Elles ne sont pas des photographies de lettres à reconnaître. La lettre est une consigne de placement donnée au système.

L'erreur quadratique moyenne, ou MSE, mesure l'écart entre les deux composantes prédites et celles du contrôleur de référence. Pour un ensemble de K exemples, $a_k^\star$ désigne la commande attendue et $\widehat{a}_k$ la commande prédite. Le calcul de la MSE est donné par l'Équation (1.1).

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{2K} \sum_{k=1}^{K} \|\widehat{a}_k - a_k^\star\|^2. \tag{1.1}$$

Les grandes erreurs pèsent davantage parce que les écarts sont élevés au carré. La MSE sert à ajuster les paramètres. Elle ne compte pas les collisions qui peuvent apparaître plus tard dans une trajectoire.

La matrice de confusion, la précision et le rappel servent à évaluer des catégories. Ici, nous utilisons la MSE pour l'imitation et quatre mesures de trajectoire, à savoir l'erreur de formation, le temps de stabilisation, les contacts et la réussite. Ces mesures complètent l'évaluation de la commande apprise.

1.6.1 Organisation des données

Le mot « donnée » recouvre plusieurs niveaux. Un épisode est une mission complète. À chaque instant, chacun des robots fournit une observation. Une observation associée à la commande du contrôleur de référence devient un exemple d'apprentissage. Un seul épisode peut donc produire de nombreux exemples qui restent liés les uns aux autres.

Cette dépendance doit être prise en compte dans l'évaluation. Si l'on mélange au hasard les instants d'une même trajectoire entre entraînement et test, le modèle peut être évalué sur des états très proches de ceux qu'il a déjà vus. Séparer les missions ou les consignes avant l'optimisation donne une évaluation plus exigeante. Réutiliser une même position de départ pour plusieurs formes doit aussi être signalé.

Les données synthétiques décrivent des situations précises. Elles portent des unités, des positions, des vitesses et des commandes. Leur qualité dépend cependant du simulateur et du contrôleur de référence. Un phénomène absent de la simulation ne peut pas être considéré comme appris simplement parce que le fichier contient beaucoup de lignes.

1.6.2 Rôle de l'ensemble de validation

L'entraînement modifie les paramètres à partir des exemples prévus pour cela. La validation aide à choisir une version du modèle, par exemple l'époque qui donne le plus petit coût. Elle participe donc aux décisions de développement. Le test réservé doit rester séparé jusqu'à ce que ces décisions soient prises.

Cette distinction explique pourquoi un essai utilisé pour corriger un modèle n'est plus un test final de cette correction. Il reste utile comme diagnostic ou comme validation. Pour mesurer la version corrigée, il faut de nouveaux cas réservés. Les campagnes du mémoire suivent cette séparation tout en précisant le faible nombre de départs distincts disponibles.

1.6.3 Missions du groupe

Le regroupement réduit la dispersion du groupe. La formation cherche une disposition précise. Le déplacement collectif transporte le groupe tout en préservant cette disposition. Ces objectifs peuvent se succéder, mais réussir une étape ne garantit pas la suivante.

Le cercle de préparation appartient au premier temps de notre protocole. Il organise les positions avant la répartition finale. Le rectangle ou la lettre demandée appartient à la mission de formation. Lorsque le groupe est ensuite transporté, il faut surveiller aussi sa destination et sa déformation. Cette distinction sert à comprendre les différentes expériences proposées dans l'interface.

1.7 Conclusion partielle

Le réseau apprend une accélération à partir d'informations locales organisées par un graphe. Le chapitre suivant situe ce choix parmi les travaux de coordination et d'apprentissage.

Chapitre 2

État de l'art

2.1 Introduction

Ce chapitre situe le mémoire parmi les approches de coordination d'un groupe de robots. Le plan distingue les méthodes traditionnelles, les méthodes avancées et trois travaux connexes. Pour chacun de ces travaux, nous présentons le contexte, la méthode, les résultats retenus et une critique. Le positionnement final distingue les idées reprises de ces travaux et les points à vérifier dans nos expériences.

La comparaison porte d'abord sur les tâches et les informations. Former une courbe, aligner les vitesses et déplacer une formation autour d'un obstacle ne demandent pas exactement les mêmes décisions. De même, un contrôleur informé de toutes les positions ne travaille pas dans les mêmes conditions qu'un contrôleur limité à ses voisins. Nous évitons donc de classer ces articles à partir de scores obtenus dans des scènes différentes.

2.2 Méthodes traditionnelles

2.2.1 Règles locales et consensus distribué

Une règle locale transforme une observation proche en action. Dans une loi d'attraction, le robot se rapproche d'une cible. Dans une loi de répulsion, il s'éloigne d'un voisin trop proche. Un freinage limite les oscillations à l'arrivée. Ces termes sont simples à expliquer séparément, mais leur combinaison dépend des gains, de la dynamique et de la disposition du groupe.

Reynolds propose un modèle de déplacement collectif associant séparation, alignement et cohésion [6]. L'intérêt pour ce mémoire est la relation entre une action locale et une organisation visible du groupe. Dans notre travail, les robots doivent aussi rejoindre des places précises. Rejoindre une lettre n'est pas seulement rester proche des autres ou adopter leur vitesse.

Un protocole de consensus cherche à faire converger des informations ou des décisions vers un accord. La réservation de notre système poursuit un accord sur les places, avec une règle de priorité. Elle ne doit pas être assimilée sans justification à un consensus moyen. Il ne s'agit donc pas de calculer une moyenne des numéros de place. Ils échangent des demandes et résolvent des conflits discrets.

Cette différence aide à choisir les tests. Pour une loi de mouvement, on vérifie des

trajectoires et des distances. Pour une réservation, on vérifie que deux propriétaires ne conservent pas la même place et que les annonces de la mission précédente ne sont pas réutilisées. Les deux mécanismes coopèrent, mais leur correction se vérifie par des observations différentes.

2.2.2 Formation de formes par contrôle décentralisé

La formation demande une information géométrique. Une cible peut être décrite par une courbe, des distances relatives, une région ou des places distinctes. Les informations nécessaires à chaque robot dépendent de cette description. Dans notre approche, le tracé commun est préparé avant le déplacement, puis chaque robot reçoit une place attribuée.

Cette représentation permet de visualiser les places et de mesurer les écarts. On peut dessiner les places et calculer leur écart. Elle impose aussi un travail d'organisation. Une forme peut être correctement échantillonnée sans que les demandes de places soient résolues. La définition géométrique et le protocole d'attribution doivent donc être décrits ensemble.

Le nombre de robots intervient dès cette étape. Garder la même petite figure tout en ajoutant des unités réduit les distances entre places. À l'inverse, agrandir trop la figure peut séparer le graphe en plusieurs parties. Cette tension entre densité et communication existe avant même que le réseau soit utilisé. Elle explique pourquoi notre application contrôle la géométrie au lancement.

2.2.3 Conception explicite de la loi de contrôle

Une commande classique indique directement comment corriger un écart de position et une vitesse. Son intérêt est de distinguer l'attraction, le freinage, la répulsion et les contraintes. Elle constitue une base de comparaison utile, surtout lorsque le modèle appris cherche à reproduire ses commandes.

Elle peut toutefois rencontrer des difficultés dans certains passages. Une répulsion peut éviter un contact tout en laissant deux robots bloqués face à face. Une formation trop large peut nécessiter un détour commun que la seule réaction locale ne prépare pas. Le diagnostic doit alors distinguer une erreur de commande, une attribution non résolue et une destination géométriquement impossible.

Dans notre système, le contrôleur de référence fournit à la fois un comportement exécutable et les réponses attendues pour l'apprentissage. Sa qualité détermine une partie de la qualité des exemples. Les cas où il ne trouve pas de commande admissible sont donc comptés. Ajouter des données ne suffit pas si le contrôleur de référence gère mal certaines situations.

2.3 Méthodes avancées

2.3.1 Variantes des GNN et justification de l'agrégation

Les réseaux sur graphes calculent une représentation à partir de relations entre éléments. Scarselli et ses coauteurs présentent un modèle général pour ce type de structure [8]. Le cadre de passage de messages de Gilmer et ses coauteurs décrit plusieurs opérations de calcul et d'agrégation dans une application aux molécules [3]. Nous retenons cette représentation relationnelle, sans transférer les résultats de cette application à la robotique.

Pour un robot, l'ordre dans lequel les voisins sont rangés dans le fichier ne devrait pas modifier le mouvement. Une somme, une moyenne ou une attention suivie d'une somme peuvent satisfaire cette exigence. Les travaux sur Deep Sets donnent un cadre pour traiter les ensembles sans dépendre de leur ordre [11]. Le choix d'une agrégation reste toutefois lié à ce que le modèle doit représenter. Le degré, les distances ou les situations de proximité ne se réduisent pas toujours à la même information.

L'attention attribue des poids calculés à partir des caractéristiques des voisins [10]. Les machines ne choisissent pas consciemment les informations à échanger. C'est une opération numérique qui permet de combiner différemment les contributions. Dans notre modèle, cette opération est locale et effectuée une fois par décision. Elle ne diffuse pas instantanément toutes les informations de l'essaim.

Le nombre de couches linéaires ne permet pas non plus de compter les échanges de voisinage. Un encodeur peut contenir plusieurs transformations sans ajouter une nouvelle portée de communication. Nous décrivons donc séparément la largeur du réseau, ses modules linéaires et son nombre d'agrégations sur le graphe.

2.3.2 Apprentissage de contrôleurs décentralisés par GNN

L'utilisation d'un réseau partagé répond à un besoin pratique. Tous les robots suivent la même règle de calcul, mais à partir d'états différents. On peut alors collecter des exemples pour l'ensemble du groupe et entraîner un seul modèle. L'exécution reste locale si ses entrées sont limitées à l'état propre et au voisinage autorisé.

Tout dépend donc des informations données en entrée. Une cible propre, une direction relative et une carte globale ne sont pas des informations équivalentes. Le qualificatif « local » doit donc être justifié par la liste exacte des données reçues, et non par le seul fait que le programme appelle un réseau sur graphe.

Notre moteur de simulation connaît toutes les positions pour calculer la physique et les métriques. Il construit cependant une observation limitée pour chaque commande. Cette séparation permet de mesurer le résultat global sans fournir automatiquement cet état global au réseau. Le protocole de collecte doit conserver la même séparation.

2.3.3 Méthodes d'apprentissage

Plusieurs stratégies peuvent servir à apprendre une commande. L'imitation donne des exemples d'actions produites par un contrôleur de référence. L'apprentissage par renforcement utilise une évaluation des conséquences des actions pour améliorer une politique. L'optimisation à travers une simulation différentiable propage une dérivée à travers le déroulement du modèle physique [1]. Ces approches n'utilisent pas la même information d'apprentissage.

La version actuelle de notre travail suit l'imitation supervisée. Le simulateur produit les exemples, puis le réseau est ajusté sur les commandes enregistrées. Le simulateur sert ici à produire les exemples, sans faire passer le gradient à travers les trajectoires physiques. Nous n'utilisons pas non plus une récompense de renforcement pour entraîner les modèles présentés.

Le passage de l'exemple isolé à une suite de décisions est une difficulté de l'imitation. Les travaux de Ross et de ses coauteurs étudient ce changement de distribution des états et une collecte qui interroge l'expert sur les états visités par la politique [7]. Notre collecte de correction s'inspire de ce principe, tout en gardant son propre protocole et ses limites. Les garanties du travail cité ne sont pas revendiquées pour notre implémentation.

2.3.4 De la règle écrite à la règle apprise

La séparation entre règle écrite et réseau n'est pas toujours complète. Une architecture peut contenir une commande nominale connue et apprendre seulement une correction. C'est le choix du modèle structuré de ce mémoire. Il limite ce que le réseau doit découvrir, mais il doit être annoncé pour interpréter une réussite.

Le filtre de sécurité est une autre forme d'association. Il agit après la proposition du réseau et peut la modifier. Il faut donc comparer le système protégé à sa version sans filtre pendant le mouvement. Sans cette comparaison, on risquerait d'attribuer au réseau la qualité d'une correction classique.

Associer ces approches peut être utile. Un système peut utilement combiner calcul appris et calcul programmé. La question scientifique est de décrire cette combinaison et de ne pas présenter le résultat de l'ensemble comme la preuve d'une capacité isolée du réseau.

2.4 Quelques travaux connexes

Les trois travaux sont présentés sous forme de tableaux pour faciliter la comparaison. Chaque tableau conserve les quatre rubriques demandées, soit le contexte, la méthode, les résultats et la critique.

2.4.1 Decentralized controllers for shape generation with robotic swarms

Le Tableau 2.1 résume cette étude [4].

Tableau 2.1 – Analyse du travail connexe 1. Synthèse de lecture [4].

Rubrique	Synthèse
Contexte de l'étude et questions de recherche	Former des courbes planes par des interactions locales. Le problème est de relier la géométrie demandée aux informations disponibles pour chaque robot.
Méthodologie adoptée	Lois décentralisées utilisant les positions relatives des voisins dans une portée connue. Analyse de stabilité et de convergence pour les courbes et les conditions considérées.
Aperçu sur les résultats	Les auteurs établissent des propriétés de formation dans leur cadre et présentent des simulations. Ces résultats concernent leur modèle, pas notre commande apprise.
Critique	Référence proche de notre tâche géométrique. Une comparaison chiffrée exigerait cependant les mêmes observations, la même dynamique et les mêmes critères. Elle ne mesure pas directement l'apport d'un réseau à messages.

2.4.2 Learning Decentralized Controllers for Robot Swarms with Graph Neural Networks

Le Tableau 2.2 résume cette étude [9].

Tableau 2.2 – Analyse du travail connexe 2. Synthèse de lecture [9].

Rubrique	Synthèse
Contexte de l'étude et questions de recherche	Apprendre un déplacement coordonné de type flocking lorsque le graphe de communication change avec les positions des robots.
Méthodologie adoptée	Imitation d'un expert centralisé pendant l'apprentissage, puis exécution locale d'un GNN partagé. Des échanges successifs permettent d'exploiter des informations à plusieurs sauts.
Aperçu sur les résultats	L'étude examine le déplacement collectif et l'intérêt des informations à plusieurs sauts lorsque la portée et la vitesse changent.
Critique	Le partage des paramètres rejoint notre choix, mais la tâche et l'expert diffèrent. Notre réseau réalise une seule agrégation par décision. Les réservations propagées appartiennent à un protocole séparé.

2.4.3 End-to-end decentralized formation control using a graph neural network-based learning method

Le Tableau 2.3 résume cette étude [5].

Tableau 2.3 – Analyse du travail connexe 3. Synthèse de lecture [5].

Rubrique	Synthèse
Contexte de l'étude et questions de recherche	Commander une formation à partir de perceptions locales, en reliant le traitement des observations et les interactions entre robots.
Méthodologie adoptée	Observations LiDAR, traitement neuronal et graphe d'interactions. Les commandes sont apprises à partir d'un expert avec une collecte complémentaire d'exemples.
Aperçu sur les résultats	Les auteurs évaluent une formation triangulaire et le fonctionnement avec différents effectifs dans un simulateur.
Critique	Notre moteur fournit directement les positions et les vitesses relatives. Il ne valide donc pas la perception LiDAR. Les scènes et les mesures diffèrent. Les taux de réussite ne permettent pas un classement direct.

2.5 Positionnement

2.5.1 Lecture critique des comparaisons

Notre comparaison principale est interne au simulateur. À l'intérieur d'une campagne, nous conservons le départ, la consigne et les critères. Nous pouvons alors examiner le rôle du filtre ou constater un écart de trajectoire entre le contrôleur de référence et le réseau.

Entre deux campagnes de développement, plusieurs éléments changent parfois ensemble. Le passage au modèle structuré modifie la commande nominale, les caractéristiques, les données et la pénalité d'apprentissage. Les résultats progressent dans le domaine testé, mais ils ne permettent pas d'isoler le rôle de l'attention.

2.5.2 Informations disponibles

Les robots disposent d'une cible préparée à partir d'une mission commune. Ils n'infèrent pas la lettre demandée à partir d'une image. Le mouvement utilise l'état propre et les voisins directs. La préparation, la réservation et la planification de navigation restent décrites séparément.

Ce découpage donne un sens précis au terme « décentralisé ». Il porte sur les décisions locales et les échanges d'attribution. La définition commune de la mission et le planificateur

sur carte connue empêchent de présenter l'ensemble comme un système sans information partagée.

Le Tableau 2.4 résume les choix et les limites correspondantes. Les lignes concernent notre système et servent à lire les résultats des chapitres suivants.

Tableau 2.4 – Choix du système et portée des validations.

Élément	Choix retenu	Limite à garder en vue
Perception	État propre et voisins à portée	Mesures exactes, sans capteur physique
Attribution	Demandes locales et priorité par identifiant	Préparation classique et durée croissante
Apprentissage	Imitation d'accélérations	Dépendance aux états et au contrôleur de référence
Formation	Réseau partagé, puis version structurée	Départs contrôlés dans le test final
Navigation	Carte connue et détour classique	Contournement non appris par le réseau
Évaluation	Départs appariés et trajectoires conservées	Peu de départs distincts réservés

2.5.3 Portée de la contribution

Le mémoire ne propose pas une nouvelle théorie générale des essaims. Il construit une chaîne de simulation et d'apprentissage où les hypothèses sont visibles. Sa valeur repose sur la possibilité de retrouver les exemples, les poids et les trajectoires derrière chaque résultat.

Le système associe des idées classiques et un modèle appris. Nous cherchons à savoir où cette association fonctionne, où elle échoue et ce qui doit être modifié. Les échecs contribuent aussi à cette analyse. Le modèle brut initial produit des contacts, le premier pilote multi-effectifs ne réussit pas les grands groupes et la première version structurée reste imparfaite en validation.

2.6 Conclusion partielle

Les méthodes traditionnelles rendent les règles explicites ; les réseaux sur graphes apprennent à combiner les informations voisines. Les trois études éclairent la géométrie, l'imitation et la perception, sans fournir de scores directement comparables aux nôtres. Le chapitre suivant décrit notre méthode et les rôles du réseau et des composants classiques.

Chapitre 3

Méthodologie et solution proposée

3.1 Introduction

Ce chapitre décrit la physique, les observations, les formes, l'attribution, les modèles et leur entraînement tels qu'ils sont implémentés. Il précise aussi les critères utilisés pour déclarer une réussite.

3.2 Organisation du système

Le système consiste à construire les places de la forme, préparer le groupe, attribuer les places puis commander le déplacement. Le simulateur enregistre les états et calcule les indicateurs. Les mêmes fonctions de mesure servent aux contrôleurs classiques et appris. Cette organisation permet de changer le contrôleur sans changer la définition d'une réussite.

Le code distingue le moteur physique, les observations locales, les références géométriques, les modèles d'imitation et les expériences. L'interface permet de sélectionner un cas et de lire sa trajectoire. Son affichage n'ajoute pas de preuve de performance. Les résultats discutés dans ce mémoire proviennent des archives d'expériences.

3.3 Modèle physique

Chaque robot est représenté par un disque de rayon $r_b = 0{,}1$ m. Sa position p_i et sa vitesse v_i sont des vecteurs à deux composantes. La commande demandée est une accélération. La fonction sat_b limite la norme d'un vecteur à la valeur b tout en conservant sa direction. Un vecteur déjà sous la limite reste inchangé. Le moteur utilise un pas de temps $\Delta t = 0{,}1$ s. Ses mises à jour sont regroupées dans l'Équation (3.1).

$$\begin{aligned} a_i^t &= \mathrm{sat}_2(a_{i,\text{demandée}}^t), \\ v_i^{t+1} &= \mathrm{sat}_1(v_i^t + \Delta t\, a_i^t), \\ \widetilde{p}_i^{t+1} &= p_i^t + \Delta t\, v_i^{t+1}. \end{aligned} \tag{3.1}$$

Les accélérations sont exprimées en m/s^2 et les vitesses en m/s. La vitesse maximale est de 1 m/s ; l'accélération maximale est de 2 m/s^2. Cette mise à jour utilise la nouvelle vitesse pour calculer la position suivante.

L'espace est un carré délimité. Si le disque atteint un mur, sa position est ramenée à la limite admissible et sa vitesse dirigée vers l'extérieur est annulée. Ce traitement empêche de sortir de l'espace simulé. Le contact reste inscrit au bilan. Un épisode avec contact n'est pas déclaré réussi.

Le Tableau 3.1 rassemble les éléments de cette étape.

Tableau 3.1 – Paramètres communs aux principales campagnes décrites. Les dimensions de l'espace et de la forme varient selon l'effectif.

Paramètre	Valeur	Rôle
Pas de temps	0,1 s	Mise à jour physique
Rayon d'un robot	0,1 m	Détection des contacts
Vitesse maximale	1 m/s	Borne physique
Accélération maximale	2 m/s^2	Borne physique
Portée principale	3 m	Voisins directs
Seuil de formation	0,1 m	Critère de stabilisation
Seuil de vitesse	0,05 m/s	Critère de stabilisation
Maintien demandé	1 s	Écarter un passage fugitif

3.3.1 Pourquoi commander une accélération ?

Une consigne de vitesse remplacerait directement le mouvement courant si le modèle ne représentait pas son évolution. Avec une accélération, la commande agit d'abord sur la vitesse. Un robot qui se rapproche rapidement de sa cible doit donc freiner à temps. Cette relation rend les trajectoires plus proches d'un problème de commande avec inertie, tout en gardant un modèle simple.

Ce modèle conserve toutefois plusieurs simplifications. Il ne représente pas l'orientation d'un véhicule à deux roues ni le lien entre couple moteur et contact au sol. La saturation de norme évite seulement de demander une vitesse ou une accélération au-delà des bornes choisies. Elle ne calcule pas une consommation d'énergie et ne garantit pas l'absence de contact.

Deux distances doivent être distinguées. Le contact physique entre deux disques de rayon 0,1 m correspond à 0,2 m entre centres. La distance de sécurité de la commande est plus grande. Elle vise à réagir avant le contact. Pendant l'entraînement structuré v2, une marge encore différente est utilisée pour guider l'apprentissage. Ces valeurs n'ont donc pas toutes le même rôle.

3.4 Représentation des formes

Le catalogue contient les 26 lettres de A à Z, les dix nombres de 1 à 10 et vingt géométries. Ces dernières comprennent le cercle, l'ellipse, le carré, le rectangle, le triangle,

le pentagone, l'hexagone, l'heptagone, l'octogone, le décagone, le losange, le trapèze, le parallélogramme, l'étoile, la croix, la flèche, le demi-cercle, le cœur, la ligne et le V. Les noms O et cercle partagent un tracé ; les noms V et géométrie en V en partagent un autre. Le décompte est donc de 56 noms pour 54 géométries différentes.

Les formes sont construites par des segments et des courbes. Pour un tracé unique, les places sont réparties le long de sa longueur. Pour plusieurs traits, un échantillonnage qui favorise des points éloignés permet de couvrir l'ensemble. Le nombre de places est égal au nombre de robots. Le réseau reçoit une cible calculée à partir de ce tracé. Il ne réalise pas de reconnaissance d'image.

Avant le lancement, les places doivent être suffisamment espacées, rester connectées à la portée choisie et tenir dans l'espace. L'écart minimal de référence entre places est de 0,4 m. L'interface peut adapter uniformément les dimensions lorsqu'une combinaison est trop dense. Pour examiner la connexité, elle utilise notamment la plus longue arête d'un arbre reliant les cibles avec une longueur totale minimale. Si aucune échelle ne satisfait les contraintes, la combinaison est refusée. Ce réglage des dimensions ne réentraîne pas le réseau.

La Figure 3.1 donne douze exemples de tracés issus du catalogue du programme. Elle décrit les consignes disponibles et ne constitue pas un bilan de réussite du réseau sur ces formes.

Une augmentation de largeur peut rendre les places moins denses, mais elle change aussi leurs distances de communication. L'adaptation doit donc satisfaire les deux conditions en même temps. Le programme recherche un intervalle d'échelles admissibles ; il ne se contente pas de remplacer une erreur de densité par un agrandissement sans contrôle. Une combinaison impossible reste refusée avec une raison.

3.5 Préparation et attribution des places

3.5.1 Pourquoi une étape circulaire ?

Le cercle visible au début est une consigne de préparation prévue par le programme. Il sert à organiser les échanges avant l'attribution finale. L'anneau est dimensionné pour laisser 0,6 m entre deux places voisines. Son rayon augmente donc avec l'effectif.

Chaque identifiant correspond à une place sur cet anneau. Une phase de freinage suit le rassemblement. Les robots échangent ensuite leurs demandes en restant immobiles, puis commencent le mouvement vers la forme demandée.

Pour les expériences du modèle structuré, les départs sont déjà proches de ces positions circulaires, avec une petite perturbation de position et une rotation commune limitée. Le résultat ne doit donc pas être interprété comme une organisation réussie à partir de positions quelconques. Les profils utilisent respectivement 8, 15 et 30 s de rassemblement

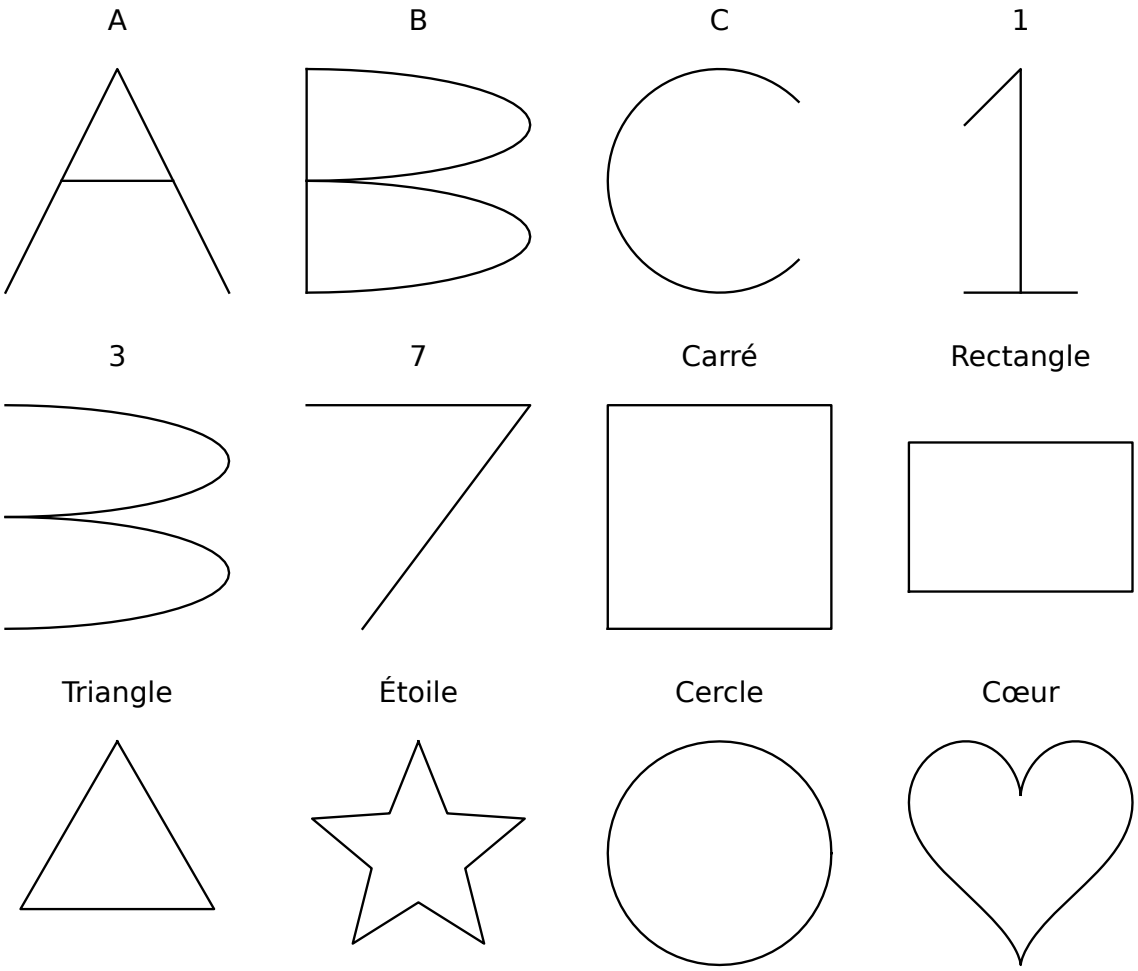


FIGURE 3.1 – Douze tracés produits par les définitions vectorielles du programme. Les paramètres d'échelle sont de 6 m par axe, sauf pour le rectangle réglé à 6 m sur 4 m. L'étendue exacte dépend du tracé. Les places sont échantillonnées ensuite selon l'effectif.

à 12, 24 et 64 robots, suivies de 2 s de freinage.

3.5.2 Réservation locale et conflits

Un robot classe les places par distance à sa position. Il annonce celle qu'il souhaite réserver et transmet les annonces connues à ses voisins. Une annonce contient un propriétaire, une version et une place. Si deux robots demandent la même place, le plus petit identifiant est prioritaire. L'autre choisit sa prochaine place disponible d'après les annonces reçues. L'identifiant sert à départager ; le réseau de neurones ne prend pas cette décision.

Passer temporairement sur une cible ne suffit pas à la réserver. Un robot peut traverser une place attribuée à un autre sans en devenir propriétaire. La commande locale doit alors limiter le risque de contact pendant le passage.

Les annonces se propagent d'un voisin à l'autre au cours de tours successifs. L'effectif est noté N. Le budget actuel est de N^2 tours, à raison de 0,1 s par tour. Il représente 14,4 s à douze robots et 409,6 s à soixante-quatre robots. Ce choix prévoit une marge importante, mais allonge l'attribution. Il correspond au protocole utilisé ici, sans représenter le nombre minimal de tours nécessaire à toute méthode décentralisée. La résolution est vérifiée avant la libération du mouvement. Lors d'un changement de mission, un nouveau numéro de mission permet d'écarter les anciennes annonces.

3.5.3 Exemple de résolution d'un conflit

Supposons que les robots 2 et 5 classent la place 3 en premier. Au départ, chacun peut annoncer sa demande sans encore connaître celle de l'autre. Lorsque les annonces se rencontrent, la priorité donne la place 3 au robot 2. Le robot 5 choisit sa prochaine place compatible avec les demandes prioritaires connues et diffuse une nouvelle version de son annonce.

Aucun centre n'a besoin de connaître les distances de tous les robots pour appliquer cette règle. Chaque robot calcule son propre classement. Les annonces transportent une réservation, pas toutes les préférences privées. Il faut toutefois du temps pour que les informations se propagent. C'est pourquoi la phase d'attribution est séparée du mouvement vers les cibles.

Le protocole ne se réduit pas à « premier arrivé, premier servi ». L'arrivée physique n'est pas la règle de priorité. Le robot peut passer près d'une place qui ne lui appartient pas et continuer vers la sienne. Une attribution obtenue avant le déplacement évite des changements permanents de cible dus aux seuls croisements de trajectoires.

À la fin de la phase, le programme vérifie l'accord et l'absence de doublons. S'il manque une réservation ou si le réseau de communication ne permet pas d'achever les échanges, la mission n'est pas déclarée résolue. Même si la forme finale paraît proche de la cible, ce problème d'organisation reste un échec.

3.6 Contrôleur de référence et protection classique

Le contrôleur de référence est une règle programmée qui produit les commandes utilisées comme exemples d'apprentissage. Il associe trois actions. L'attraction rapproche le robot de sa cible ; le freinage réduit sa vitesse ; la répulsion l'éloigne des voisins trop proches. Les gains utilisés sont respectivement 6, 4 et 60.

La protection examine ensuite la commande proposée. Elle cherche une accélération aussi proche que possible de cette proposition, tout en respectant les limites de mouvement et les contraintes locales de séparation. Le calcul est une optimisation quadratique, abrégée QP. Il s'effectue sur les deux composantes de l'accélération.

Les voisins prennent chacun une part de la réaction nécessaire. La distance de sécurité utilisée ici est de 0,4 m et le paramètre de réaction vaut 3. Une petite composante de détour aide à départager un face-à-face. Ces mécanismes ajustent la commande tout en conservant la cible attribuée.

Si aucune commande admissible n'est trouvée, le système applique un freinage borné et compte cette situation. Le filtre ne garantit donc pas la sécurité dans toutes les situations. Les contacts restent mesurés sur toute la trajectoire.

En navigation, la protection tient aussi compte des obstacles et des murs. L'obstacle étant immobile, la correction revient entièrement au robot mobile. Le trajet commun et la protection locale ont ainsi des rôles complémentaires.

3.7 Premier modèle à attention sur graphe

3.7.1 Entrées et calcul de la commande

Le premier modèle, nommé GAT initial dans la suite, reçoit huit valeurs propres qui décrivent la position, la vitesse, la cible, la phase et l'effectif. Le prétraitement combine l'écart à la cible divisé par 6 m, la vitesse, la phase divisée par trois et l'effectif divisé par douze. Il fournit ainsi six valeurs. Chaque voisin apporte sa position et sa vitesse relatives. Leur distance complète ces quatre composantes pour former cinq caractéristiques d'arête.

Deux couches linéaires avec activation ReLU encodent l'état propre ; deux autres encodent les arêtes. Les représentations ont une largeur de 64. Des projections de requête et de clé donnent les scores d'attention. La somme pondérée des représentations voisines est combinée avec l'état propre, l'écart à la cible, la vitesse et le degré du robot. Deux couches de sortie donnent l'accélération, ramenée à la norme maximale si nécessaire.

Le modèle contient ainsi huit modules linéaires et une seule agrégation de voisinage par décision. Aucun objectif privé d'un voisin ne lui est fourni. Les poids sont partagés entre tous les robots.

3.7.2 Du fichier d'observation au calcul local

L'observation propre est rangée dans un vecteur de huit valeurs. Les voisins forment une petite matrice dont chaque ligne contient quatre valeurs relatives. Comme le nombre de voisins varie, le calcul par lots utilise un masque. Celui-ci indique quelles lignes correspondent à de vrais voisins et quelles lignes servent seulement à compléter la taille du tableau.

Les lignes masquées ne doivent influencer ni l'attention ni la somme finale. Le code vérifie ce comportement. Sans lui, l'ajout de lignes vides pour constituer un lot pourrait changer artificiellement une accélération. Un voisinage vide reçoit également un traitement explicite pour éviter une division ou une normalisation indéfinie.

Le lot d'apprentissage regroupe des observations de robots, pas une décision centrale pour l'essaim entier. Le réseau applique le même calcul à chacune de ces observations. Le moteur réunit ensuite les commandes pour mettre à jour l'état physique commun. L'entraînement et l'exécution utilisent ainsi la même définition des caractéristiques locales.

3.7.3 Lecture des dimensions de l'architecture

Le Tableau 3.2 donne les dimensions des principaux calculs. Le GAT initial et le GAT structuré contiennent chacun huit modules linéaires, mais leurs entrées et leurs sorties intermédiaires diffèrent. Le tableau ne représente pas huit tours de communication.

Tableau 3.2 – Dimensions des blocs des deux modèles.

Bloc	GAT initial	GAT structuré
État propre prétraité	6 valeurs	4 valeurs
Encodage propre	6 vers 64, puis 64 vers 64	4 vers 48, puis 48 vers 48
Caractéristiques voisines	5 valeurs par voisin	8 valeurs par voisin
Encodage voisin	5 vers 64, puis 64 vers 64	8 vers 48, puis 48 vers 48
Requête et clé	Deux projections de 64 vers 64	Deux projections de 48 vers 48
Sortie apprise	133 vers 64, puis 64 vers 2	96 vers 48, puis 48 vers 3 coefficients
Commande finale	Deux composantes bornées	Base et corrections vectorielles bornées

Dans le premier modèle, les deux nombres de sortie sont directement les composantes de la proposition d'accélération. Dans le second, les trois coefficients calculés pour une arête pondèrent des directions déjà définies. Ils ne correspondent ni à trois classes de formes ni à trois moteurs physiques.

Le contrôle de base du modèle structuré suit une forme proportionnelle et dérivée, dite PD pour *Proportional-Derivative*. Lorsque la cible est fixe, le terme proportionnel réagit à l'écart de position et le terme dérivé freine la vitesse. Les gains sont appris sur

des exemples isolés, puis conservés pendant l'apprentissage des corrections. Cette partie explicite explique pourquoi nous parlons d'un modèle structuré.

Le tableau permet de séparer trois questions. La largeur décrit la taille d'une représentation. Le nombre de modules décrit les transformations apprises. Le nombre d'agrégations décrit l'utilisation du voisinage. Présenter ces trois valeurs évite de résumer l'architecture par un seul nombre de « couches » qui serait ambigu.

3.7.4 Modes brut et protégé

Le mode protégé applique le filtre classique à l'accélération du réseau. Le mode brut retire cette projection pendant le rassemblement et le mouvement vers la forme, c'est-à-dire les phases apprises. Les bornes physiques restent appliquées par le simulateur. Le freinage et l'attribution restent des étapes classiques communes. Le mot « brut » porte donc sur la commande des phases de mouvement ; il ne signifie pas que toute la mission serait réalisée par le réseau.

3.8 Données et entraînement du modèle initial

3.8.1 Origine et séparation des données

Le jeu de données est produit par le contrôleur de référence. Une trajectoire fournit plusieurs observations robot par robot et instant par instant. Seuls les exemples des phases de mouvement apprises sont retenus pour l'imitation. La séparation est faite avant l'optimisation et tient compte des formes équivalentes. Deux noms du même tracé restent dans la même partition.

La collecte élargie contient 214 épisodes du contrôleur de référence, répartis en 185 fichiers d'entraînement, 20 de validation et 9 de test. Après regroupement des tracés identiques, cela représente 180 trajectoires d'entraînement, 18 de validation et 9 de test distinctes. Les géométries uniques sont réparties en 36 pour l'entraînement, 9 pour la validation et 9 pour le test. Les observations retenues comprennent 531 360 exemples d'entraînement distincts et 53 136 de validation.

Il faut distinguer ce que représentent ces nombres. Un exemple est une observation locale, pas une nouvelle mission indépendante. Cinq graines de départ sont réutilisées entre les formes d'entraînement, deux entre celles de validation et une entre celles de test. La collecte contient donc huit configurations initiales distinctes, répétées pour différentes consignes. Cette réutilisation facilite les comparaisons mais limite la diversité des départs.

Le Tableau 3.3 précise les différentes unités de la collecte. Les fichiers portant des noms de formes équivalentes restent rattachés à leur partition, mais leur répétition n'ajoute pas une nouvelle géométrie ni une nouvelle trajectoire distincte.

Tableau 3.3 – Partitions de la collecte élargie du modèle initial.

Partition	Noms	Géométries	Fichiers	Trajectoires distinctes
Entraînement	37	36	185	180
Validation	10	9	20	18
Test	9	9	9	9

Le contrôle des doublons a un but précis. Si le cercle était rangé en entraînement et la lettre O en test alors que leurs tracés sont identiques, la réserve ne mesurerait pas une nouvelle géométrie. Le regroupement des formes équivalentes empêche cette confusion. Il ne crée pas pour autant une grande diversité des positions de départ.

Les états voisins dans le temps sont aussi liés. Le robot observé à 10,0 s et à 10,1 s peut avoir presque la même position. On conserve ces exemples pour apprendre le mouvement, mais on ne les compte pas comme deux missions indépendantes. La taille du fichier indique le nombre d'exemples. Les graines et les épisodes donnent une meilleure idée de la diversité des situations.

3.8.2 Contenu d'une observation enregistrée

Le Tableau 3.4 précise le contenu brut utilisé pour construire un exemple. Les huit valeurs propres ne sont pas toutes transmises telles quelles à l'encodeur : chaque modèle applique le prétraitement décrit plus haut. Le tableau distingue donc le fichier d'observation de la représentation interne.

Tableau 3.4 – Contenu d'un exemple local avant prétraitement.

Information	Taille	Sens
Position propre	2 nombres	Coordonnées du centre, en mètres
Vitesse propre	2 nombres	Mouvement sur les deux axes, en m/s
Cible attribuée	2 nombres	Place destinée au robot, en mètres
Phase et effectif	2 nombres	Étape courante et taille du groupe
Un voisin	4 nombres	Position et vitesse relatives
Masque des voisins	Variable	Lignes présentes dans le calcul
Commande attendue	2 nombres	Accélération de référence, en m/s^2

La commande attendue est disponible pendant l'apprentissage, mais elle ne fait pas partie de l'entrée du réseau à l'exécution. Il serait sinon possible de recopier la réponse. De même, les objectifs privés des voisins ne sont pas ajoutés à leurs caractéristiques relatives.

Le masque répond à un besoin de rangement des lots. Si deux observations possèdent des nombres de voisins différents, certaines lignes peuvent être complétées pour former un tableau commun. Ces lignes sont ignorées. Leur présence ne doit pas changer le poids d'un

vrai voisin ou produire une correction fictive.

Les unités restent celles du simulateur. Le prétraitement ramène certaines grandeurs à des échelles fixes pour le calcul neuronal. Il ne transforme pas les mètres de l'expérience en une nouvelle unité physique. Les commandes et les métriques exportées restent interprétables dans les unités annoncées.

3.8.3 Optimisation et sélection

L'optimiseur Adam utilise un taux d'apprentissage de 5×10^{-4} et des lots de 1 024 exemples. La graine d'initialisation vaut 42. Le budget maximal est de 80 époques, avec arrêt après douze époques sans amélioration en validation. La MSE est définie par l'Équation (1.1). Son unité est le m^2/s^4, puisqu'elle porte sur des écarts d'accélération au carré.

L'entraînement s'arrête après 23 époques. Les poids retenus correspondent à l'époque 11, avec une MSE de validation de 0,112516. Le test réservé est ensuite exécuté avec ces poids fixes. Les exemples de test sont réservés à l'évaluation finale, après l'ajustement des paramètres et le choix de l'époque.

3.8.4 Déroulement d'un entraînement

Le programme charge d'abord les observations et les commandes de la partition d'entraînement. Pour chaque lot, le modèle produit ses deux composantes d'accélération. La fonction de coût mesure l'écart avec la commande du contrôleur de référence. L'optimiseur ajuste ensuite les paramètres pour réduire cet écart. Une époque correspond à un passage sur les exemples sélectionnés.

Après l'époque, la validation est calculée sans mettre à jour les paramètres. Si son coût s'améliore, une copie des poids est conservée. L'arrêt anticipé évite de poursuivre indéfiniment lorsque les nouvelles époques n'améliorent plus la sélection. Les poids de la dernière époque ne sont donc pas nécessairement ceux du modèle retenu.

L'évaluation en trajectoire intervient ensuite. Le simulateur appelle le modèle à chaque décision et les erreurs peuvent modifier les états suivants. Ce test observe un comportement différent du calcul d'une moyenne sur des exemples fixes. Les résultats de ces deux évaluations sont conservés. La courbe d'apprentissage explique la sélection, les trajectoires décrivent la mission.

Reprendre un entraînement change les poids. Rejouer une expérience avec des poids déjà sauvegardés ne les change pas. L'interface utilise cette seconde opération lorsqu'elle affiche un résultat ; choisir une nouvelle forme ou une nouvelle graine n'entraîne pas le réseau à nouveau.

3.9 Corrections et modèle structuré

3.9.1 Collecter sur les états rencontrés par le réseau

Les contacts du premier modèle brut ont motivé une collecte de correction. Le contrôleur de référence est interrogé sur des états visités par le modèle, uniquement pour construire les données d'apprentissage prévues. Les situations proches d'un voisin reçoivent un poids plus élevé. Les trajectoires ratées restent conservées ; les demandes pour lesquelles le contrôleur de référence ne trouve pas de correction admissible sont comptées et exclues des étiquettes utilisables.

Cette opération réduit une partie du décalage entre les exemples initiaux et les états rencontrés pendant l'exécution. Elle n'a cependant pas suffi à rendre le premier réseau brut fiable. Un pilote entraîné avec plusieurs effectifs a également montré que la capacité à recevoir un nombre variable de voisins ne garantissait pas la réussite à 24 ou 64 robots. Ces observations ont conduit à revoir la structure de la commande.

3.9.2 Mouvement nominal et corrections vectorielles

Le GAT structuré contient une commande de base qui rapproche le robot de sa cible tout en le freinant. La cible du robot i est notée g_i. La commande de base est définie par l'Équation (3.2).

$$a_i^{\text{base}} = \text{sat}_2\left(k_p(g_i - p_i) - k_d v_i\right), \qquad k_p > 0, \quad k_d > 0. \tag{3.2}$$

Les deux gains positifs sont appris sur 8 192 exemples synthétiques sans voisin, puis fixés pour la suite. Ils retrouvent les valeurs 6 et 4. Le réseau apprend alors des corrections liées au voisinage au lieu de devoir réapprendre seul le mouvement nominal.

Le réseau calcule ensuite une correction à partir des voisins. Elle combine un éloignement, un déplacement de côté et une réaction à leur vitesse relative. L'attention règle le poids de chaque contribution. Un facteur lié à l'activité annule la correction lorsque le robot est à sa cible et que le voisinage est immobile.

L'Équation (3.3) résume l'ajout de cette correction, notée c_i, à la commande de base.

$$\widehat{a}_i = \text{sat}_2\left(a_i^{\text{base}} + c_i\right). \tag{3.3}$$

Cette écriture regroupe le calcul des messages dans un seul terme. Elle ne remplace pas le calcul du programme. Sans voisin, la correction est nulle.

Le réseau calcule ses coefficients à partir de distances, de normes et de projections sur les directions radiale et tangente. Son encodage propre comporte quatre valeurs et son encodage d'arête huit valeurs. Il utilise une largeur de 48, l'activation SiLU, huit

modules linéaires et deux gains positifs appris. Cette construction suit les translations et les rotations du plan. Si la scène tourne, les vecteurs de commande tournent avec elle. Le choix d'un sens de détour empêche d'étendre cette affirmation sans précaution aux réflexions.

3.9.3 Préparation des exemples et coût d'apprentissage

Les données locales issues du pilote multi-effectifs sont complétées par des voisinages synthétiques. Les effectifs représentés sont 12, 24 et 64, avec différents nombres de voisins et des situations de proximité. Après sélection, le jeu utilisé contient 30 000 exemples d'entraînement et 21 000 de validation, dont respectivement 12 000 et 3 000 voisinages ajoutés. Les candidats dont la commande du contrôleur de référence n'est pas admissible sont comptés avant exclusion, soit 100 en entraînement et 28 en validation pour cette augmentation.

L'objectif combine une erreur d'imitation pondérée et une pénalité lorsque la commande viole les inégalités locales de séparation. Une écriture du principe est : L'Équation (3.4) sépare les deux contributions du coût.

$$\mathcal{L} = \mathcal{L}_{\text{imitation,pondérée}} + \lambda\, \mathcal{L}_{\text{séparation}}. \tag{3.4}$$

Le coefficient λ règle le poids de la pénalité de séparation. La seconde partie pénalise les commandes qui ne respectent pas les contraintes locales de séparation. Elle est normalisée par la distance et guide l'apprentissage. Elle guide l'apprentissage, sans filtrer les mouvements du modèle brut.

Une première version réussit six des neuf cas de validation. La seconde repart de ses poids, augmente le poids de séparation à 200 et utilise une distance d'apprentissage de 0,5 m. Le taux d'apprentissage est de 3×10^{-4}, la taille des lots de 512 et le budget maximal de 60 époques, avec une patience de douze. L'entraînement s'arrête à l'époque 53 ; l'époque 41 est retenue. La valeur de validation est de 0,239340 pour cet objectif composite. Elle ne peut pas être comparée directement à la MSE du premier modèle, ni à une valeur calculée avec une autre pénalité.

La Figure 3.2 présente les courbes issues des deux journaux d'entraînement.

3.10 Navigation avec une carte connue

Le déplacement d'une formation entière utilise un centre de référence mobile. Le contrôleur reçoit l'état dans le repère de ce centre. La position et la vitesse de référence sont soustraites à l'état propre. L'accélération de référence est ensuite ajoutée à la commande locale. Les positions relatives entre voisins restent celles de la scène physique.

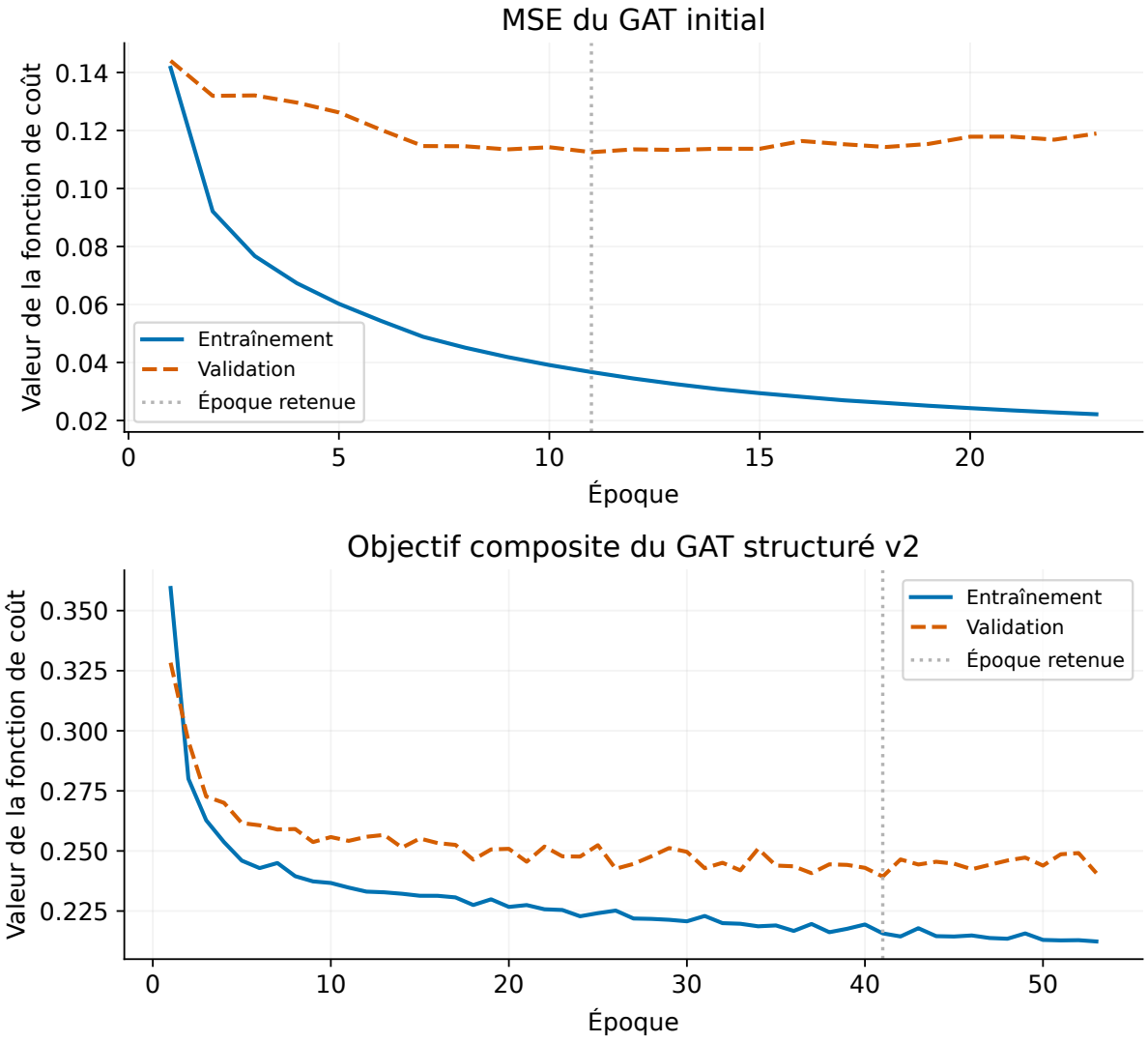


FIGURE 3.2 – Courbes extraites des journaux d'entraînement. En haut, l'imitation du modèle initial est mesurée par une MSE. En bas, le modèle structuré v2 utilise un coût composite. Les deux ordonnées n'ont pas la même signification.

Le trajet est calculé par une recherche classique sur une grille de 0,1 m, puis simplifié en segments libres. Le planificateur tient compte de toutes les places de la formation. Un passage libre pour le seul centre peut rester trop étroit pour les robots situés sur les côtés. Pour chaque obstacle, il examine donc les positions interdites du centre de translation en fonction des décalages des cibles. Une marge de planification de 0,35 m s'ajoute aux rayons physiques.

Chaque segment est parcouru avec une progression lisse qui démarre et se termine à vitesse nulle. La durée est réglée pour limiter la vitesse de référence à 0,2 m/s et ne descend pas sous quatre secondes. Une période de stabilisation complète le transport. La forme n'est pas tournée ou déformée par cette recherche. Une cible finale occupée par un obstacle est refusée avant le calcul, plutôt que déplacée silencieusement.

Dans les scènes finales à plusieurs obstacles, la préparation est classique pour les deux contrôleurs comparés. Le transport utilise ensuite soit le contrôleur de référence, soit le GAT initial protégé. Le GAT structuré brut n'est pas évalué dans ces scènes. Le réseau ne reçoit pas la carte complète et n'apprend pas le planificateur.

3.11 Critères d'évaluation

3.11.1 Erreur de formation et maintien

Pour les positions $P = \{p_i\}$ et les cibles $Q = \{q_j\}$, l'erreur de couverture est : L'erreur de couverture est définie dans l'Équation (3.5).

$$E(P, Q) = \max\left(\max_i \min_j \|p_i - q_j\|, \max_j \min_i \|q_j - p_i\|\right). \tag{3.5}$$

Elle mesure la plus grande distance non couverte entre les deux ensembles. Contrairement à une moyenne, elle ne masque pas un robot très éloigné parce que les autres seraient bien placés. Elle ne dépend pas de l'ordre des robots dans les tableaux.

La stabilisation demande simultanément $E(P, Q) \leq 0{,}1$ m et une vitesse maximale de 0,05 m/s pendant une seconde. Avec le pas retenu, cette fenêtre inclut onze états échantillonnés. Le temps de stabilisation correspond à la première fenêtre admissible selon le protocole. L'erreur finale est mesurée au dernier état. Ces deux indicateurs ne décrivent donc pas exactement le même moment.

3.11.2 Contacts et réussite

Les contacts entre disques sont recherchés aussi entre deux états successifs, en examinant le déplacement interpolé des centres. Un contact prolongé n'est pas recompté comme un nouvel événement à chaque image. Les contacts initiaux et les nouvelles entrées en contact sont conservés. Pour la navigation, les obstacles et les murs font également partie du bilan.

Un épisode de formation réussit si l'attribution est résolue, si la stabilisation est atteinte avant la fin et si aucun contact n'a eu lieu pendant toute l'expérience. Une bonne forme finale après un contact reste donc un échec. Lorsque la stabilisation n'est pas atteinte, aucun temps de réussite n'est indiqué.

La navigation demande aussi une erreur de destination inférieure à 0,1 m et une déformation maximale pendant le transport inférieure à 0,2 m. La mesure de forme est recentrée sur la formation, tandis que l'erreur de destination mesure le placement du groupe. Cela distingue un groupe bien formé au mauvais endroit d'un groupe arrivé mais déformé.

3.12 Reproductibilité et lecture des résultats

Une graine permet de reproduire un tirage de positions initiales. À configuration identique, changer de contrôleur sans changer cette graine donne une comparaison appariée. La graine sert à reproduire le départ, et non à apprendre ou à mesurer la qualité du modèle. Si plusieurs formes utilisent le même départ, elles ne deviennent pas pour autant plusieurs départs indépendants.

Les archives conservent les positions, les vitesses, les commandes, les instants et les paramètres. Les rapports donnent les métriques et les empreintes des poids utilisés. Les tableaux et les figures du chapitre suivant sont reconstruits à partir de ces fichiers. Les tests logiciels vérifient les fonctions du programme ; les campagnes de trajectoires évaluent les comportements. Cette distinction reste valable même lorsque tous les tests logiciels réussissent.

3.12.1 Calcul et environnement enregistré

Le journal du GAT initial indique une exécution sur le processeur central, ou CPU pour *Central Processing Unit*. Il enregistre PyTorch 2.15.0+cpu et NumPy 2.5.4. Ces informations concernent ce journal précis. Elles ne constituent pas une mesure de vitesse du réseau sur toutes les machines.

Un processeur graphique, ou GPU pour *Graphics Processing Unit*, peut servir à effectuer certains calculs d'apprentissage en parallèle. Nos résultats ne sont toutefois pas présentés comme une comparaison CPU/GPU. Les temps du chapitre d'expériences sont des temps simulés de mission, et non des durées de calcul matériel.

Le nom du logiciel ne suffit pas pour reproduire une expérience. Il faut retrouver les poids, les paramètres physiques, la consigne, l'état de départ et le protocole. Le même numéro de graine avec une autre taille de forme ne décrit pas le même cas. Inversement, une lecture plus rapide de l'animation ne modifie pas la trajectoire déjà calculée.

3.12.2 Formats des fichiers et provenance

Le format JSON, pour *JavaScript Object Notation*, conserve des informations structurées comme les paramètres et les résumés de métriques. Le format CSV, pour *Comma-Separated Values*, permet de lire les trajectoires comme un tableau. L'export de démonstration indique notamment l'instant, le robot, ses coordonnées et ses deux composantes de vitesse.

Les tableaux numériques complets sont aussi conservés dans des archives NumPy portant l'extension `.npz`. Cette extension est un format de fichier, pas une nouvelle métrique. Le paquet d'export rassemble les données nécessaires à la lecture du cas ; ses fichiers doivent être conservés ensemble pour garder leur contexte.

Une empreinte SHA-256, issue de la famille *Secure Hash Algorithm*, identifie le contenu d'un fichier. Elle permet de vérifier que les poids examinés sont bien ceux utilisés pour le résultat. Elle identifie le fichier, mais ne mesure pas la qualité du modèle ou de sa trajectoire. Elle ne sert pas non plus à cacher les données.

Pour les figures du mémoire, le PDF, ou *Portable Document Format*, conserve les tracés vectoriels. Les images PNG, pour *Portable Network Graphics*, sont exportées à 300 points par pouce, notés DPI pour *Dots Per Inch*. Le catalogue possède aussi une sortie SVG, pour *Scalable Vector Graphics*. Ces formats concernent la présentation et l'archivage, sans ajouter de résultats d'apprentissage.

À l'impression, la couleur est accompagnée de repères de forme. Les courbes ont des traits distincts, les barres sont hachurées et les positions finales sont marquées par des cercles vides. Le lecteur peut ainsi suivre les catégories en niveaux de gris. La vérification visuelle complète celle des valeurs, sans changer les critères de réussite.

3.13 Configurations actuelles et progression de l'apprentissage

Le passage à plusieurs effectifs modifie aussi la taille de la scène. Le Tableau 3.5 donne les profils utilisés pour le test structuré. Il évite d'attribuer au seul nombre de robots des écarts produits par des configurations plus larges et des préparations plus longues.

Tableau 3.5 – Profils contrôlés des campagnes multi-effectifs.

Robots	Arène (m)	Largeur (m)	Rassemblement (s)	Freinage (s)	Allocation (s)
12	10	6	8	2	14,4
24	18	12	15	2	57,6
64	32	28	30	2	409,6

La progression suivie n'est pas un apprentissage cumulatif de toutes les lettres une à une. Le développement passe par le modèle initial, la collecte de correction, le pilote

multi-effectifs, puis le modèle structuré. Chaque changement possède ses données et ses poids. Les résultats de l'un ne sont pas ajoutés au taux de réussite d'un autre pour annoncer un score global.

Le deuxième modèle structuré repart des poids de sa première version, mais modifie la pénalité et la marge utilisées pendant l'apprentissage. La valeur du coût n'est donc pas directement comparable entre ces deux réglages. La réussite en validation est examinée avant de figer la version soumise au test réservé.

3.14 Organisation logicielle et niveau de preuve

Le Tableau 3.6 présente les principales parties du programme. Les noms indiquent où retrouver les calculs décrits, sans confondre le moteur, l'apprentissage et l'affichage.

Tableau 3.6 – Organisation des composants de la version actuelle.

Partie	Fonction	Données produites ou utilisées
`src/simulator/`	Physique et observations	États, commandes, voisinages
`src/reference/`	Formes, attribution, contrôleur de référence, navigation	Cibles, annonces, trajets
`src/imitation/`	Collecte, modèles et entraînement	Exemples, poids, journaux
`demo/`	Expériences et affichage web	Réglages, animation, exports
`tasks/evidence/`	Rapports conservés	Résumés, audits, empreintes

Trois vérifications sont complémentaires. Les tests du logiciel contrôlent des opérations prévues. Les campagnes mesurent le comportement des contrôleurs. Les audits relisent les archives pour vérifier la cohérence physique et les métriques annoncées. Même nombreux, les tests du logiciel ne remplacent pas l'évaluation du modèle sur de nouveaux cas.

3.15 Conclusion partielle

La méthode sépare la forme, l'attribution des places et le mouvement. Les deux modèles apprennent à partir de commandes synthétiques, puis sont évalués sur des trajectoires complètes. Le chapitre suivant présente l'implémentation et les résultats des campagnes.

Chapitre 4

Implémentation, expérimentations et résultats

4.1 Introduction

Ce chapitre rassemble les résultats enregistrés pendant le développement. Les cas ratés sont présentés avec les cas réussis, car ils expliquent les changements apportés au modèle et les limites de la solution finale.

4.2 Ce que fait l'application

L'application prépare une mission à partir d'une forme, d'un effectif, d'un contrôleur et d'un départ reproductible. Elle vérifie d'abord la compatibilité géométrique. Elle calcule ensuite les mouvements et affiche la trajectoire. Le temps affiché sous l'animation est le temps simulé de la mission ; le temps nécessaire au calcul sur l'ordinateur est une autre durée.

Le menu propose trois contrôleurs et deux comparaisons. Le contrôleur classique utilise les règles programmées. Le GAT protégé ajoute un filtre à la proposition du premier réseau. Le GAT structuré brut utilise le second modèle pendant les phases apprises, dans le domaine de formation prévu. Les deux autres choix lancent chacun une paire d'expériences pour faciliter la comparaison.

Ce menu ne représente donc pas cinq réseaux entraînés. Il faut lire le nom du contrôleur réellement exécuté avant d'interpréter l'animation. Une formation classique à soixante-quatre robots ne démontre pas une performance neuronale. L'affichage d'une simulation avec un effectif exploratoire ne suffit pas à valider ce réglage.

Le cercle initial correspond au rendez-vous du protocole. Le numéro reproductible change le tirage de départ, avec des variations limitées autour de l'anneau pour le modèle structuré. Il ne change pas la définition du réseau. La lecture complète conserve le freinage et les échanges de places, même si le bouton de résultat final permet d'aller directement au dernier état.

Le résultat peut être exporté avec les positions, les vitesses, les paramètres et les métriques. Ces fichiers permettent une lecture indépendante de l'interface. Une erreur visible doit pouvoir être reliée à un instant et à une mesure, plutôt qu'à la seule impression donnée par la vitesse de lecture.

4.2.1 Conditions d'une démonstration reproductible

Pour reproduire un cas, on conserve ensemble la graine, l'effectif, la forme, ses dimensions, la scène et le modèle. La graine doit donc être conservée avec les autres réglages. Une modification de largeur ou de contrôleur suffit à changer la trajectoire, même si le numéro de départ reste identique.

Le serveur local se lance depuis la racine du projet avec la commande suivante. Elle ouvre le service de démonstration ; elle ne lance pas un entraînement :

```
.\demo \start -reference.ps1
```

La page est ensuite accessible à l'adresse `http://127.0.0.1:8765/`. Les commandes détaillées, les domaines des contrôleurs et les exports sont décrits dans le guide d'expériences conservé avec le projet. Les scores du présent chapitre restent ceux des campagnes archivées, pas ceux d'une démonstration improvisée.

4.3 Protocole des campagnes

Les expériences répondent à trois questions. Le premier modèle peut-il reproduire les commandes du contrôleur de référence tout en réalisant une formation sans contact ? Le changement de structure du modèle permet-il d'utiliser une commande brute avec plusieurs effectifs ? Enfin, la chaîne hybride de navigation peut-elle déplacer une formation autour d'obstacles proches de ses cibles ?

Les campagnes sont présentées dans leur ordre de développement, car leurs échecs expliquent les choix suivants. Elles n'utilisent pas toutes les mêmes données ni les mêmes conditions. Une amélioration entre deux campagnes ne suffit donc pas à attribuer un gain à un seul composant. À l'intérieur d'une campagne, les contrôleurs comparés partent des mêmes états et utilisent les mêmes critères de réussite.

Les valeurs proviennent des rapports conservés dans le dépôt. Les figures montrent des trajectoires enregistrées et des courbes issues des journaux d'entraînement. Leurs légendes précisent la campagne, le contrôleur et le cas représenté.

Le Tableau 4.1 rassemble les éléments de cette étape.

4.3.1 Même départ, même critère

La comparaison appariée sert à éviter qu'un contrôleur soit avantagé par un départ plus facile. À forme et scène identiques, le contrôleur de référence et le réseau reçoivent les mêmes états initiaux. On vérifie aussi que la trajectoire n'est pas remplacée par un état final préparé à l'avance.

Une campagne reste toutefois liée à son domaine. La réserve du modèle initial utilise des formes différentes de celles de l'entraînement, mais peu de départs. Le test structuré

Tableau 4.1 – Principales campagnes discutées. « Cas » désigne une combinaison de forme, de départ et d'effectif ou de scène ; il ne signifie pas toujours un départ distinct.

Campagne	Question principale	Ensemble évalué
Référence multi-formes	Effet du filtre classique	9 formes réservées, 12 robots
Correction	Effet d'exemples sur les états du réseau	18 cas frais, 12 robots
Pilote multi-effectifs	Passage à 24 et 64 robots	3 formes par effectif
Modèle structuré v2	Mouvement appris sans filtre	18 cas réservés, 12/24/64 robots
Navigation finale	Transport près des obstacles	8 cas par contrôleur, carré à 12 robots

utilise trois consignes et deux départs par effectif. La navigation utilise une scène définie par le programme. Ces ensembles ne constituent pas un échantillon aléatoire de tous les environnements possibles.

4.4 Ce que montre l'entraînement

Les courbes du chapitre précédent montrent une baisse du coût d'entraînement pour les deux modèles. Pour le réseau de référence, la validation atteint son meilleur niveau avant la fin. Continuer les époques réduit encore l'erreur sur les exemples appris sans améliorer le choix de validation. C'est la raison du retour aux poids de l'époque retenue.

Pour le réseau structuré, l'ordonnée regroupe l'imitation et la pénalité de séparation. Une valeur plus grande que la MSE du premier réseau ne signifie donc pas un modèle moins bon. Comparer ces nombres sans préciser leurs termes reviendrait à comparer deux mesures différentes.

La bonne lecture consiste à suivre une courbe à l'intérieur de son propre réglage, puis à examiner le mouvement avec les critères communs. Les résultats de trajectoire donnent un verdict plus directement lié à la mission. Ils peuvent aussi révéler un échec malgré un faible coût sur les exemples enregistrés.

4.5 Limites du premier modèle d'imitation

Le GAT initial est entraîné avec douze robots à partir des commandes du contrôleur de référence. Après sélection des poids sur la validation, neuf formes sont évaluées dans une partition réservée. Les résultats sont présentés dans le Tableau 4.2. La même configuration initiale est utilisée pour ces formes ; les neuf épisodes ne représentent donc pas neuf tirages de départ indépendants.

Tableau 4.2 – Campagne réservée du modèle initial. Les erreurs et temps sont des moyennes ; le temps porte sur les épisodes réussis. Le tiret indique qu'aucun épisode n'a réussi. Données issues du rapport 008d.

Contrôleur	Réussites	Contacts	Erreur finale (m)	Temps (s)
Contrôleur de référence	9/9	0	0.00000	29.83
GAT initial brut	0/9	54	0.01269	–
GAT initial protégé	9/9	0	0.01269	30.43

Le contrôleur de référence réussit les neuf cas sans contact. Le réseau protégé réussit également les neuf, avec une erreur finale moyenne d'environ 1,27 cm. Son temps moyen de stabilisation est de 30,43 s, contre 29,83 s pour le contrôleur de référence. Ces temps comprennent la préparation et l'attribution.

Le réseau brut atteint presque la même erreur finale moyenne, mais provoque 54 événements de contact et ne réussit aucun cas. Cet écart est le résultat principal de la campagne. La dernière image, prise seule, aurait donné une impression trompeuse du résultat. Les robots peuvent finir près des places demandées après s'être touchés pendant le mouvement. Le filtre classique a donc une contribution importante à la réussite du système protégé.

La MSE de validation, égale à 0,112516 pour les poids retenus, mesure l'imitation de commandes sur des exemples. Elle ne remplace pas cette évaluation du déplacement. Les états rencontrés par le réseau peuvent aussi s'éloigner progressivement de ceux du contrôleur de référence, même si les premières erreurs de commande sont modestes.

4.5.1 Un cas qui explique le rôle du filtre

L'ellipse à douze robots, avec la graine 7300, appartient aux neuf cas réservés. Elle montre pourquoi nous examinons le déplacement avant de conclure à partir de la dernière image. Le Tableau 4.3 donne les mesures de ce cas précis ; il ne s'agit pas d'une nouvelle campagne.

Tableau 4.3 – Cas de l'ellipse, douze robots, graine 7300.

Contrôleur	Erreur finale (m)	Contacts	Temps de réussite (s)
Référence	0.00000	0	26.6
GAT initial brut	0.00928	5	–
GAT initial protégé	0.00928	0	28.9

Le GAT brut et le GAT protégé finissent tous les deux à environ 9,28 mm de la forme. Le premier compte pourtant cinq événements de contact, contre aucun pour le second. Le critère de réussite porte sur l'épisode complet. Le temps de réussite du brut reste donc absent, même si sa dernière disposition est proche des places demandées.

La Figure 4.1 suit l'erreur et la séparation minimale pendant ce même épisode. La distance entre centres est calculée sur les déplacements interpolés, afin de ne pas regarder seulement les images échantillonnées. Le seuil physique de contact vaut 0,2 m pour les disques utilisés.

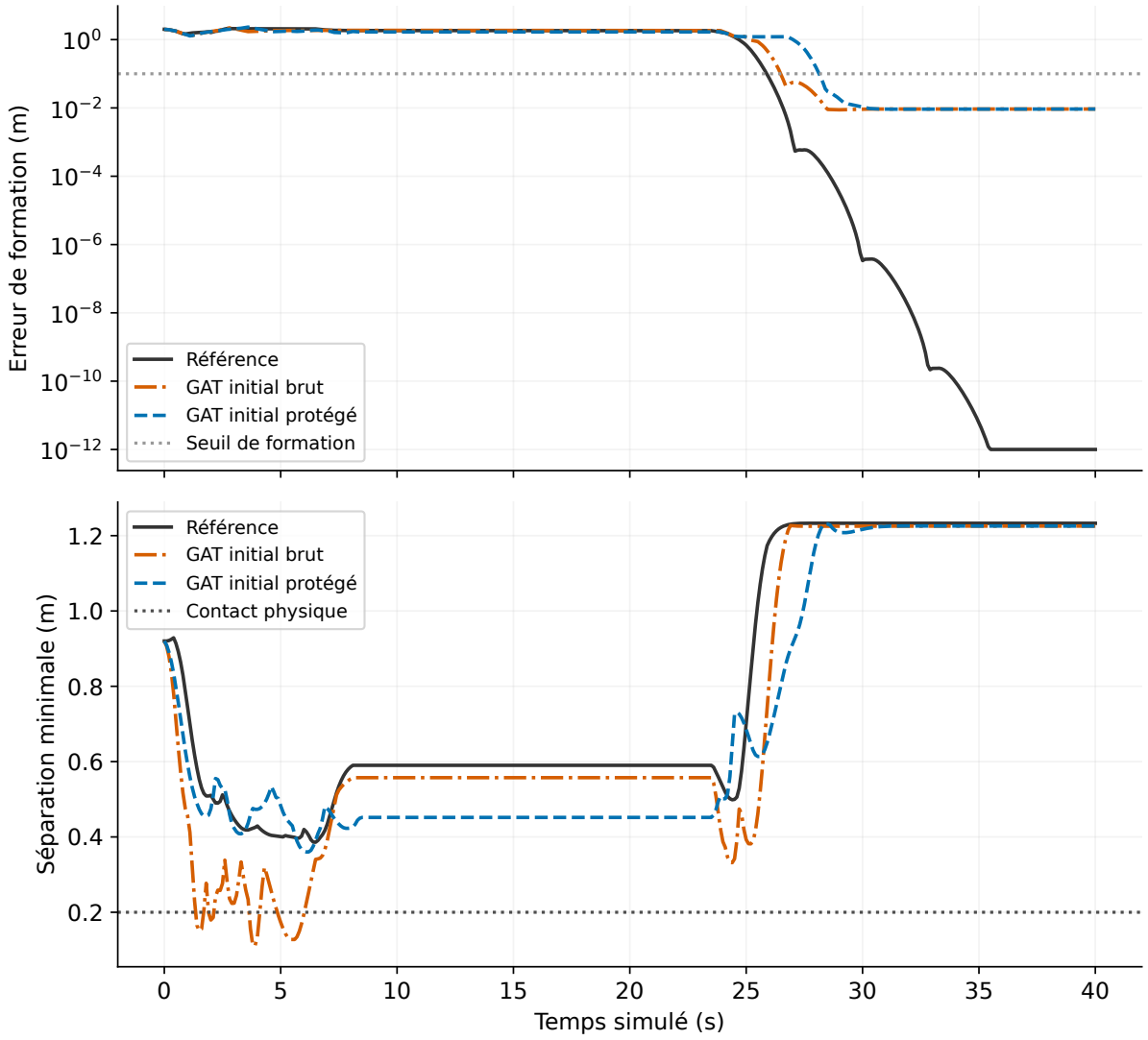


FIGURE 4.1 – Ellipse à douze robots, graine 7300. En haut, l'erreur de couverture. En bas, la séparation minimale des centres sur les intervalles interpolés. Les traits distinguent les trois commandes en niveaux de gris. Pour le tracé logarithmique, les erreurs inférieures à 10^{-12} m sont affichées à ce seuil. Données issues de la réserve 008d.

Le filtre ne change pas la cible de cet essaim. Il modifie certaines propositions d'accélération pendant le mouvement. Ce cas illustre donc son rôle dans la chaîne protégée. Il ne suffit pas, à lui seul, à établir un taux de réussite général et ne doit pas être ajouté une seconde fois aux neuf cas de la campagne.

4.6 Résultats de la collecte de correction

Une collecte complémentaire interroge le contrôleur de référence sur des états rencontrés par le modèle. Le nouvel entraînement donne davantage de poids aux situations proches d'un voisin. L'évaluation suivante utilise neuf formes et deux graines fraîches, soit 18 cas.

Sur ces mêmes cas, le modèle brut initial provoque 102 contacts. Le modèle corrigé en provoque 65, soit une baisse d'environ 36,3 %. Les deux restent à zéro réussite sur dix-huit. La version corrigée avec protection réussit les dix-huit cas sans contact.

La correction a donc un effet mesurable, mais elle n'atteint pas l'objectif d'un mouvement brut sans contact. Cette version ne peut donc pas être considérée comme fiable. Les poids corrigés et les trajectoires ratées sont conservés pour expliquer cette conclusion, tandis que le modèle initial protégé reste disponible pour la démonstration hybride.

La Figure 4.2 rassemble les contacts de cette comparaison. La baisse entre les deux commandes brutes est mesurée sur les mêmes dix-huit cas. Le troisième résultat inclut une protection. Il ne mesure pas seulement l'effet des nouvelles données.

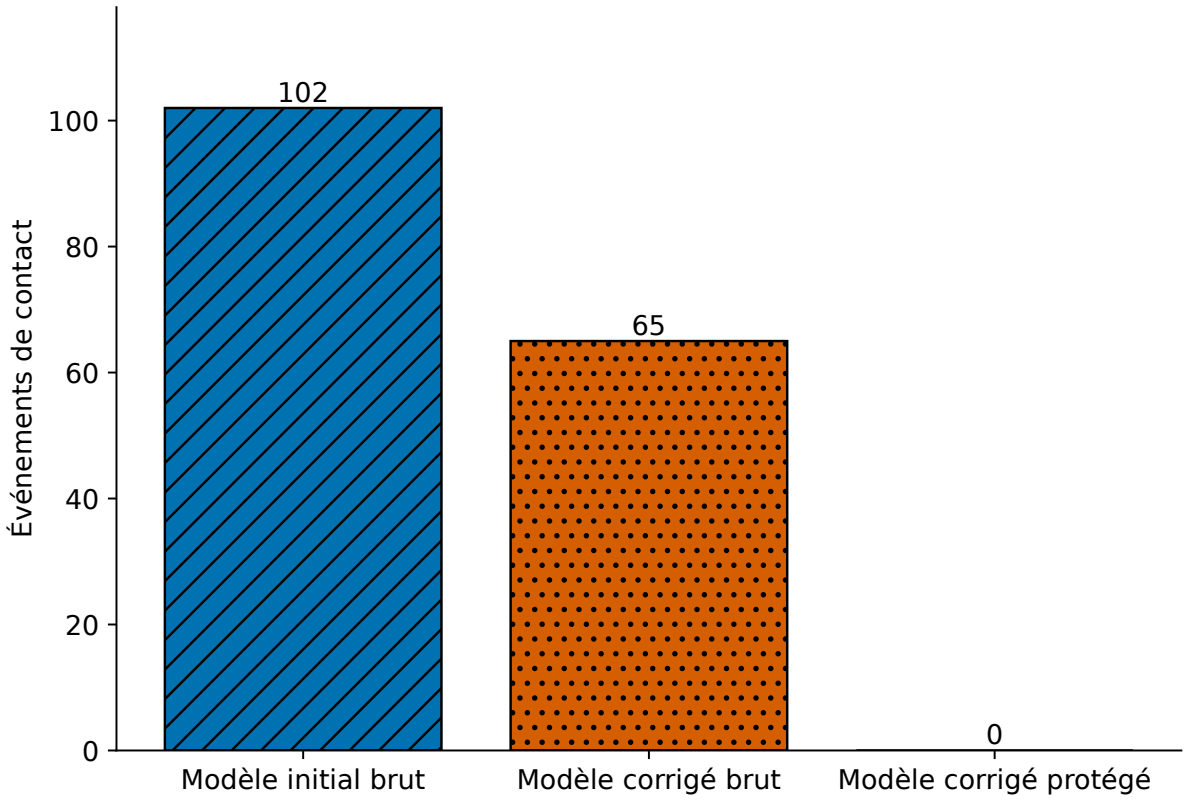


FIGURE 4.2 – Événements de contact dans la campagne de correction pour le modèle initial brut, le modèle corrigé brut et le modèle corrigé protégé. Les dix-huit cas sont communs aux trois commandes. Données issues du rapport 011.

Le nombre de contacts et le nombre d'épisodes réussis n'ont pas la même unité. Une trajectoire peut compter plusieurs contacts ; réduire leur somme ne suffit pas si chaque

épisode en conserve au moins un ou ne satisfait pas une autre condition. Le modèle brut progresse, mais ne remplit toujours pas les conditions de réussite.

4.7 Premier essai avec plusieurs effectifs

Le pilote multi-effectifs utilise 12, 24 et 64 robots. Les dimensions de l'espace, de la forme et de la préparation sont adaptées à chaque profil. Trois formes sont réservées au test pour chaque effectif. Les résultats sont :

Le Tableau 4.4 rassemble les éléments de cette étape.

Tableau 4.4 – Pilote multi-effectifs avant le modèle structuré. Données issues du rapport 012.

Robots	Réussites brutes	Contacts bruts	Réussites protégées	Contacts protégés
12	2/3	1	3/3	0
24	0/3	65	0/3	0
64	0/3	127	0/3	0

À 24 et 64 robots, le système protégé évite les contacts dans ces cas mais ne termine pas correctement le protocole. Le rassemblement ne permet pas d'aboutir à l'attribution attendue. L'absence de collision ne suffit donc pas à réussir la mission.

Ce pilote montre aussi qu'un réseau qui accepte un nombre variable de voisins n'est pas automatiquement performant avec un effectif plus grand. Les interactions, les dimensions et la durée des phases changent. La bonne valeur de la perte de validation du pilote ne résout pas ces difficultés en exécution.

4.8 Formation à plusieurs effectifs avec le modèle structuré

4.8.1 Validation avant le test réservé

La première version structurée réussit six des neuf cas de validation, avec six contacts dans les trois cas restants. Ces échecs ne sont pas écartés du bilan. L'entraînement de la seconde version renforce la pénalité de séparation et utilise une marge plus grande. Cette seconde version réussit ensuite les neuf cas de validation, portant sur le triangle, la lettre B et le nombre 7 aux trois effectifs.

Les poids sont alors fixés avant l'exécution du test réservé. Ce test utilise le rectangle, la lettre C et le nombre 3, avec les graines 8500 et 8501 à chacun des effectifs 12, 24 et 64. Cela donne dix-huit cas, mais seulement six configurations de départ distinctes, avec

deux départs par effectif réutilisés pour trois formes. La conclusion statistique doit rester proportionnée à cette diversité.

4.8.2 Résultats du test réservé

Le Tableau 4.5 rassemble les résultats du GAT structuré brut. Dans les dix-huit cas, aucun contact n'est observé et aucun filtre de sécurité n'intervient pendant les phases de mouvement apprises. Le contrôleur de référence, évalué sur les mêmes départs, réussit lui aussi les dix-huit cas sans contact.

Tableau 4.5 – Test réservé du GAT structuré v2. Les erreurs et temps sont des moyennes sur six cas par effectif. Les temps incluent toutes les phases de préparation. Données issues du rapport 014.

Robots	Réussites	Contacts	Erreur finale (m)	Temps GAT (s)	Temps réf. (s)
12	6/6	0	1.79×10^{-7}	29.43	30.00
24	6/6	0	3.44×10^{-7}	85.23	85.23
64	6/6	0	8.67×10^{-7}	465.80	465.93

Les erreurs finales sont très faibles, de l'ordre de 10^{-7} à 10^{-6} m. Elles sont obtenues après une longue stabilisation dans une simulation sans bruit. Ces valeurs ne décrivent pas la précision que l'on pourrait attendre de robots réels. Pour la mission, le critère utile reste le respect du seuil de 0,1 m, du seuil de vitesse, du maintien et de l'absence de contact.

L'audit des 36 trajectoires, dix-huit pour chaque contrôleur, vérifie les mises à jour physiques, les bornes, les contacts, l'attribution, la fenêtre de stabilisation et l'identité des départs comparés. Il vérifie aussi que les fichiers utilisent les poids retenus. La plus petite distance entre centres observée pendant le déplacement interpolé est d'environ 0,546 m à douze robots, 0,516 m à vingt-quatre et 0,411 m à soixante-quatre. La distance de 0,5 m choisie pendant l'apprentissage n'est donc pas toujours maintenue. L'absence de contact dans ces cas ne constitue pas une garantie générale sur cette marge.

La Figure 4.3 montre quatre états enregistrés d'une formation à 24 robots.

4.8.3 Pourquoi le temps augmente-t-il autant ?

Le temps moyen atteint 465,8 s à 64 robots. Il s'agit du temps simulé depuis le début de la mission, et non du temps de calcul d'une décision du réseau. Le budget d'attribution vaut à lui seul 409,6 s à cet effectif. Les phases de rassemblement et de freinage ajoutent 32 s, et le mouvement vers la forme commence à 441,6 s. À 24 robots, il commence à 74,6 s ; à 12 robots, à 24,4 s.

La Figure 4.4 distingue les durées de préparation et de mouvement.

Cette lecture change l'interprétation du résultat. Le réseau ne met pas plus de quatre cents secondes à calculer une accélération. Le protocole attend de nombreux

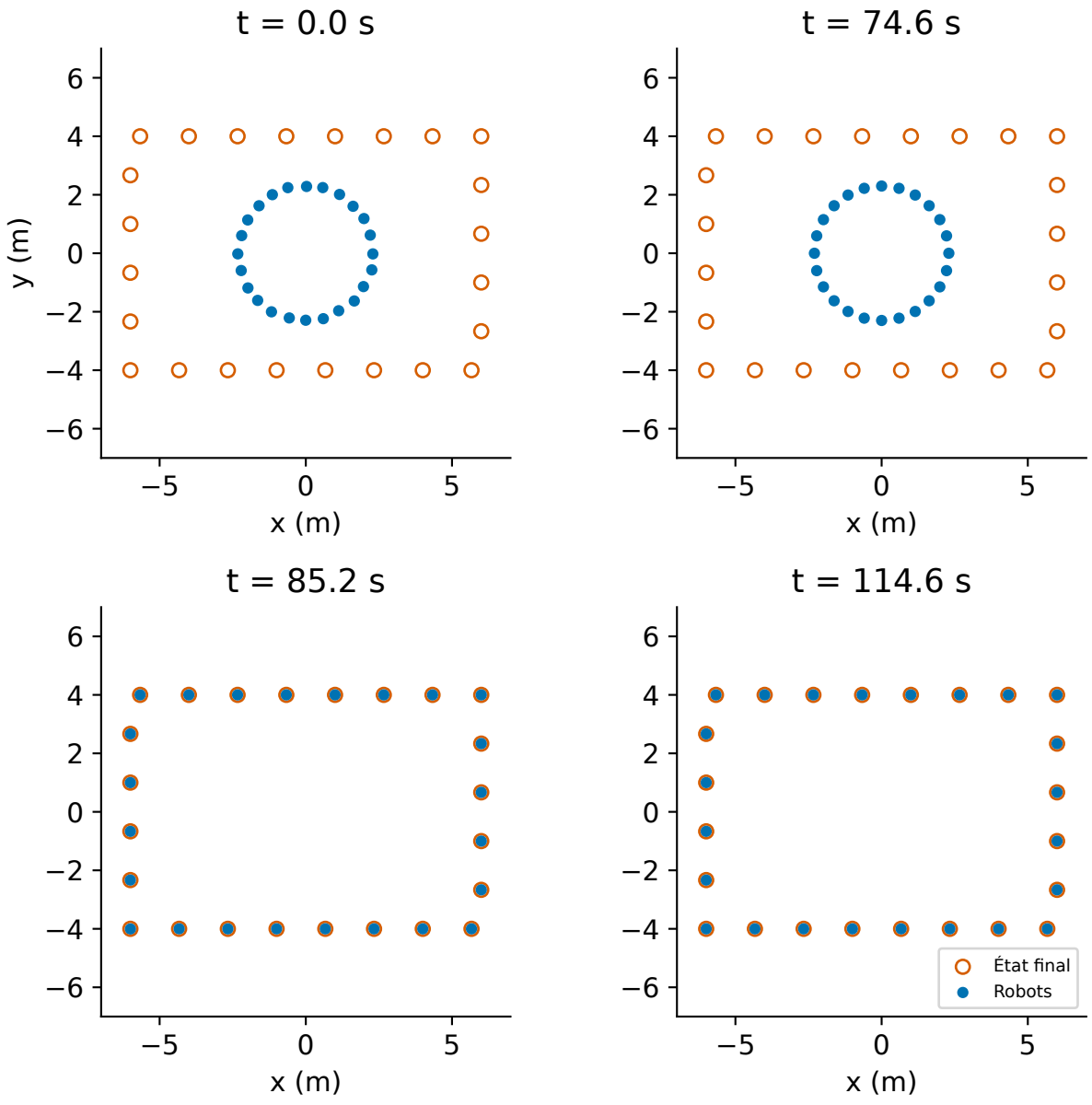


FIGURE 4.3 – Quatre états d'une trajectoire enregistrée du rectangle à 24 robots, graine 8500, GAT structuré brut. Les cercles vides indiquent les positions finales enregistrées, les points pleins les robots à l'instant indiqué. Le cercle initial est une étape du protocole.

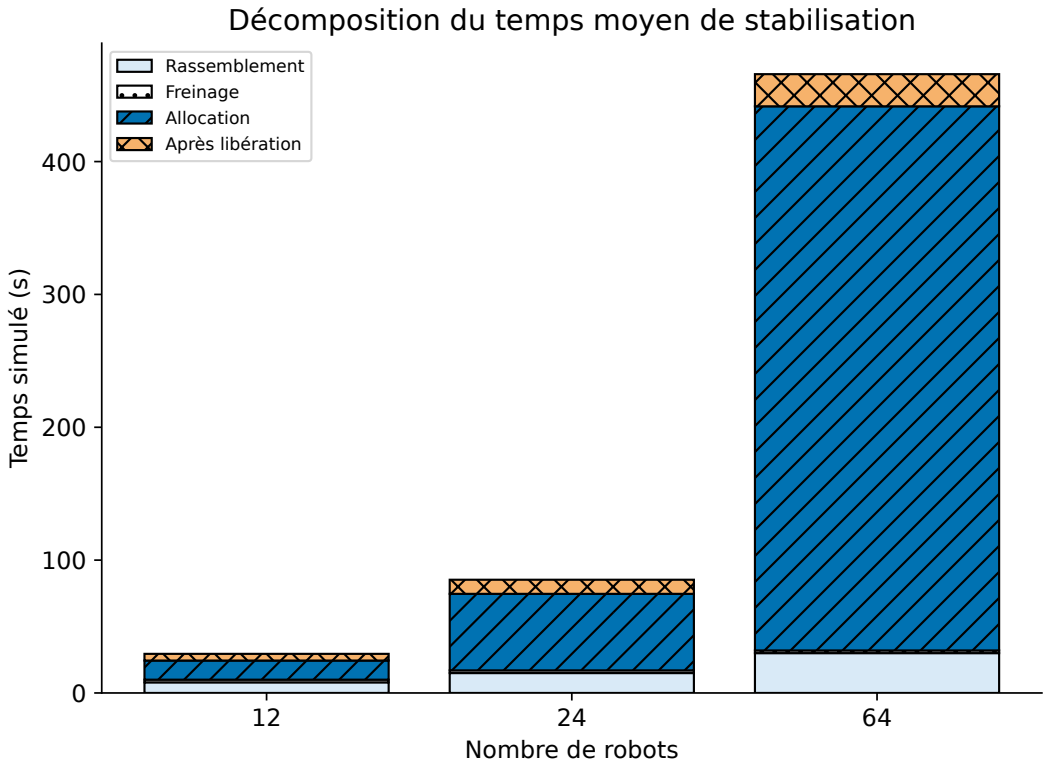


FIGURE 4.4 – Temps de stabilisation décomposé à partir du protocole et des moyennes mesurées du GAT structuré. L'allocation domine à 64 robots. La dernière partie couvre le temps entre la libération du mouvement et la stabilisation.

tours d'échanges avant de lancer le mouvement final. Réduire ce budget demanderait un travail sur l'attribution et sa condition d'arrêt, puis une nouvelle vérification des trajectoires.

4.8.4 Lecture des dix-huit cas

Le Tableau 4.6 détaille les temps de stabilisation du modèle structuré et du contrôleur de référence. Le bilan moyen ne vient donc pas d'une sélection des meilleures trajectoires. Chaque combinaison annoncée de forme, d'effectif et de graine est présente.

Les temps voisins du contrôleur de référence et du réseau ne constituent pas une preuve de supériorité. Les deux réussissent le critère dans cette réserve et partagent les longues phases de préparation. L'écart utile à examiner serait aussi le temps après libération du mouvement, avec davantage de départs et plusieurs entraînements.

La Figure 4.5 montre cette partie du mouvement pour le rectangle à la graine 8500. Les courbes sont alignées sur la libération, afin de ne pas comparer les attentes d'attribution. Le seuil d'erreur et le seuil de vitesse sont tracés séparément ; leur franchissement doit encore être maintenu pour satisfaire le protocole.

Ces trois courbes sont des exemples parmi les cas du tableau. Elles n'ajoutent pas de nouveaux départs à la réserve. Leur intérêt est de rendre visible le lien entre une mesure

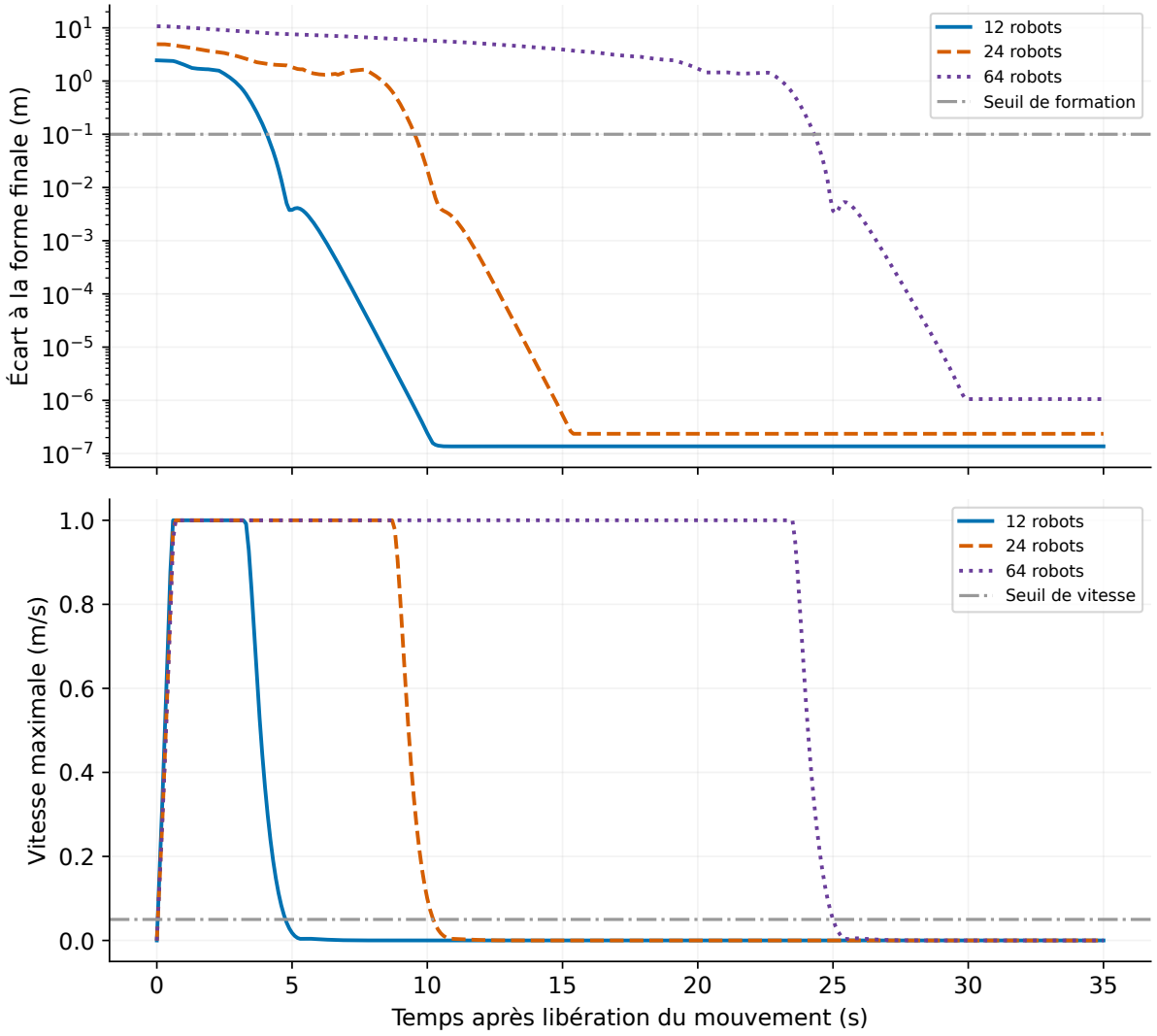


FIGURE 4.5 – Évolution enregistrée après libération du mouvement du rectangle, graine 8500, aux trois effectifs. L'erreur est calculée par rapport aux cibles attribuées et représentée en échelle logarithmique ; la seconde courbe montre la vitesse maximale. GAT structuré brut.

Tableau 4.6 – Détail des cas réservés du modèle structuré v2.

Robots	Forme	Graine	Temps GAT (s)	Temps réf. (s)	Contacts GAT
12	Rectangle	8500	29.2	29.5	0
12	C	8500	27.9	29.7	0
12	3	8500	31.2	31.7	0
12	Rectangle	8501	29.2	29.5	0
12	C	8501	27.9	27.9	0
12	3	8501	31.2	31.7	0
24	Rectangle	8500	84.9	84.7	0
24	C	8500	84.1	84.0	0
24	3	8500	86.7	87.0	0
24	Rectangle	8501	84.9	84.7	0
24	C	8501	84.1	84.0	0
24	3	8501	86.7	87.0	0
64	Rectangle	8500	466.6	466.0	0
64	C	8500	462.9	462.8	0
64	3	8500	467.9	469.0	0
64	Rectangle	8501	466.6	466.0	0
64	C	8501	462.9	462.8	0
64	3	8501	467.9	469.0	0

finale et le mouvement qui y conduit.

4.9 Transitions entre formes

Une mission peut demander plusieurs formes successives. Le système conserve alors les positions et les vitesses entre les étapes. Il ne replace pas les robots directement sur la forme suivante. Les annonces d'attribution sont renouvelées pour la nouvelle mission, et l'enregistrement permet de vérifier cette continuité.

Les campagnes de transitions du GAT initial protégé et du contrôleur de référence comprennent un pilote de six séquences puis une validation de deux séquences par contrôleur. Les séquences validées réussissent sans contact. Les huit étapes présentes dans les deux séquences de validation ne sont pas huit expériences indépendantes. L'état de chaque étape dépend de la précédente. Ces résultats concernent les contrôleurs évalués dans cette campagne, et non une validation supplémentaire du GAT structuré brut.

4.10 Navigation et obstacles proches des cibles

4.10.1 Difficulté et correction du trajet

Les premières expériences de navigation ont rencontré des cas sans stabilisation malgré l'absence de contact. Le déplacement du seul centre ne tenait pas suffisamment compte

de la largeur de la formation. Un autre cas demandait de placer un robot sur une cible occupée par un obstacle. Il fallait préparer un détour dans le premier cas et refuser la destination impossible dans le second.

Le planificateur corrigé vérifie le déplacement de toutes les places autour des obstacles. Une campagne ultérieure sur trois formes et des parcours complémentaires réussit huit cas par contrôleur, sans contact. Une dernière campagne rapproche ensuite les obstacles des cibles et conserve les mêmes exigences de réussite. Les résultats de cette dernière campagne sont détaillés ci-dessous ; ils ne remplacent pas rétroactivement les échecs des scènes antérieures.

4.10.2 Campagne finale

Les essais finaux portent sur un carré de largeur 6 m à douze robots. Le centre de référence se déplace de $(-1, 0)$ à $(1, 0)$ m. Les scènes à plusieurs obstacles contiennent de cinq à dix disques. Les centres des obstacles sont situés à au plus un mètre d'une place finale du carré. Leur disposition est définie dans le programme ; il ne s'agit pas de cartes tirées au hasard dans un environnement inconnu.

La campagne comporte les six nombres d'obstacles, de cinq à dix, avec la graine 8901 ; elle ajoute dix obstacles avec la graine 8902 et un cas à un obstacle avec cette même graine. Cela représente huit cas pour le contrôleur de référence et huit pour le GAT initial protégé. Pour les sept cas à plusieurs obstacles, la préparation est classique et commune ; la différence de contrôleur concerne le transport. Le cas à un obstacle vérifie également le comportement de la chaîne déjà disponible.

Le Tableau 4.7 rassemble les éléments de cette étape.

Tableau 4.7 – Campagne finale avec obstacles proches des cibles. Les deux dernières colonnes sont les maxima entre les cas, et non des moyennes. La déformation est mesurée pendant le transport. Données issues du rapport 019.

Contrôleur	Réussites	Contacts	Déformation max. (m)	Destination max. (m)
Contrôleur de référence	8/8	0	$3{,}06 \times 10^{-11}$	$1{,}11 \times 10^{-16}$
GAT initial protégé	8/8	0	0,010388	0,005550

Les huit cas réussissent pour chacun des deux contrôleurs. L'audit de seize trajectoires confirme la physique et les métriques. Le GAT protégé reste à environ 1,04 cm de déformation maximale et 0,56 cm d'erreur maximale de destination. Ces résultats portent sur une chaîne hybride qui associe un trajet classique sur carte connue, une préparation classique dans les scènes multiples, une commande locale et un filtre. Ils ne permettent pas de conclure que le réseau a appris, à lui seul, à contourner dix obstacles.

La Figure 4.6 montre les trajectoires de transport et les obstacles de la scène finale.

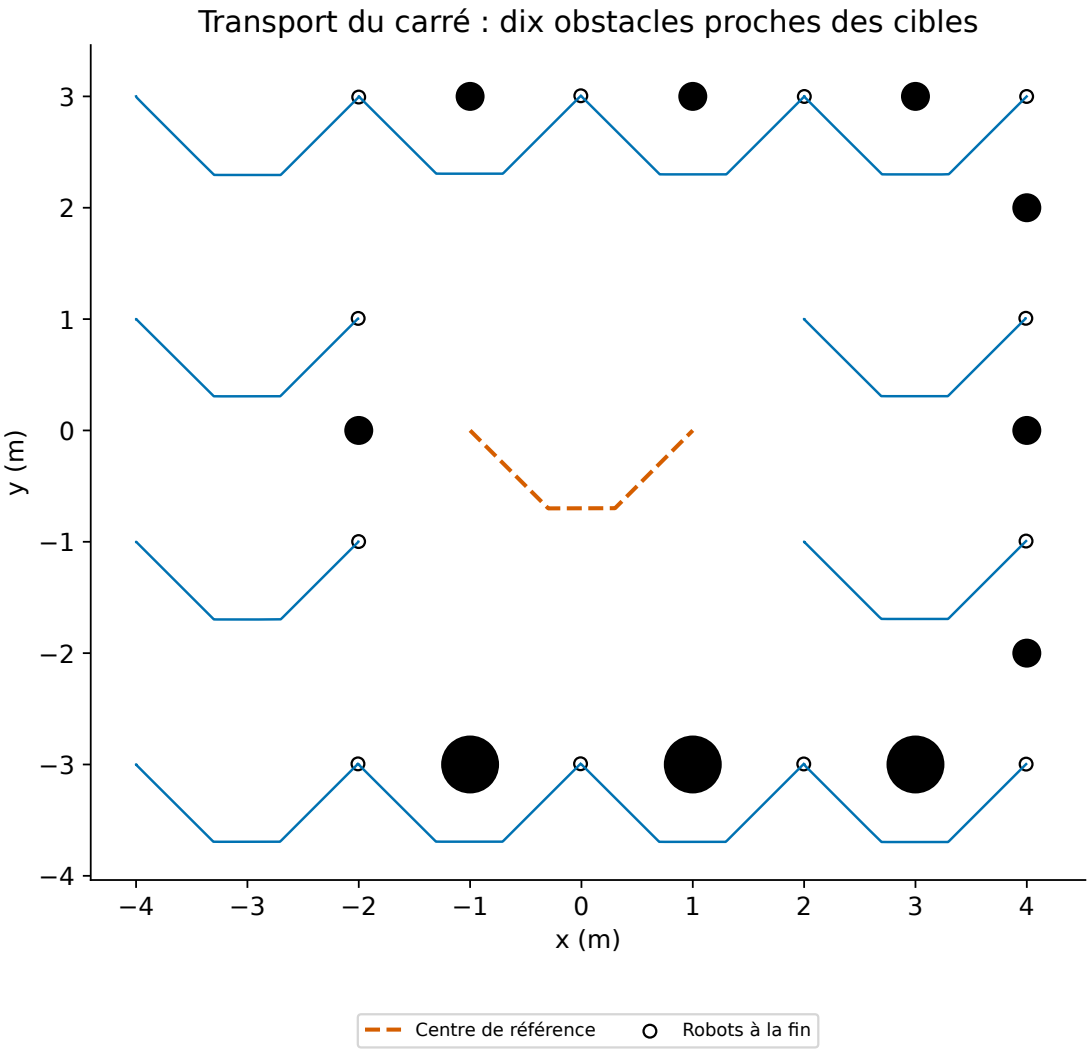


FIGURE 4.6 – Trajectoires enregistrées pendant le transport du carré avec dix obstacles, graine 8901, GAT initial protégé. Les disques noirs représentent les obstacles à leur rayon physique. Les traits continus suivent les centres des robots ; les cercles vides indiquent leur état final. Le trait discontinu suit le centre de référence. La préparation n'est pas tracée.

4.11 Analyse des métriques et des résultats négatifs

Quatre situations doivent être distinguées à la lecture. Une grande erreur finale indique que le groupe n'occupe pas la forme attendue. Une petite erreur avec une vitesse élevée indique que les robots n'y sont pas encore stabilisés. Une petite erreur et une faible vitesse après un contact restent un échec de l'épisode. Enfin, une formation correcte mais trop loin de la destination peut rater une mission de transport.

Le message de résultat doit renvoyer à cette cause. « Stabilisation non atteinte » ne signifie pas nécessairement qu'une fonction du logiciel a cessé de répondre. Il signifie que les conditions prévues n'ont pas été maintenues à temps. À l'inverse, une exception de lecture ou une image non dessinée relève de l'interface et doit être corrigée sans changer le verdict physique.

Le cas d'une cible bloquée est encore différent. Il est détecté avant l'exécution lorsque l'obstacle occupe une place finale. Déplacer cette place pour faire réussir la mission changerait la consigne. Le programme conserve donc la demande et refuse ce cas, au lieu de le compter parmi les transports réussis.

4.12 Lecture d'ensemble

Trois constats se dégagent. La MSE ne suffit pas à prévoir la réussite, le modèle structuré atteint l'objectif dans sa réserve contrôlée et la navigation reste hybride. Les campagnes antérieures conservent leurs échecs ; les scènes corrigées ont leurs propres archives et les mêmes critères.

Les expériences ne suffisent pas à isoler l'effet de l'attention. Plusieurs éléments ont changé entre le premier réseau et le modèle structuré, notamment la représentation, la commande nominale, les données et la fonction de coût. Une comparaison qui retire uniquement les messages, répétée avec plusieurs entraînements et de nouveaux départs, reste nécessaire pour mesurer cet apport séparément.

Certaines combinaisons proposées par l'interface restent à évaluer. Les effectifs 8 et 16 et les autres formes ou dimensions restent exploratoires selon le modèle. Une exécution classique à 64 robots ne prouve pas une performance du réseau. Le contrôleur et son domaine doivent accompagner chaque démonstration.

4.13 Conclusion partielle

Les essais établissent une réussite du modèle structuré dans son domaine contrôlé et une réussite de la navigation hybride dans les scènes finales. Ils montrent aussi les limites du premier modèle brut et du pilote multi-effectifs. La discussion suivante examine la portée de ces constats.

Discussion

Introduction

La discussion revient sur la question posée au début du mémoire. Dans quelles conditions une commande locale apprise peut-elle former un essaim sans contact, et quel rôle les composants classiques conservent-ils ? Les résultats apportent une réponse partielle. Ils établissent des réussites dans des domaines précis, tout en montrant plusieurs échecs avant la version structurée. Cette lecture demande de distinguer l'imitation, la mission complète et l'étendue de la validation.

Le dernier résultat ne doit pas faire oublier les difficultés rencontrées. Le premier modèle brut, la collecte de correction et le pilote multi-effectifs montrent des limites différentes. Ces limites expliquent les choix de conception. Elles aident aussi à préparer une démonstration honnête. Le nom du contrôleur, le départ et les composants actifs doivent accompagner le score.

Interprétation des résultats

Le premier résultat concerne l'écart entre une bonne imitation et une mission réussie. Le réseau de référence atteint une faible erreur finale sur les formes réservées, mais sa version brute produit des contacts. Le filtre classique change ce bilan. Les neuf cas réussissent lorsqu'il protège la commande. Cette réussite repose donc sur le réseau et le filtre réunis.

Il n'y a pas de contradiction entre une petite erreur finale et un épisode raté. L'erreur de formation décrit une disposition à un instant. Le bilan de contact décrit un événement qui peut avoir eu lieu plus tôt. Notre définition de réussite exige une disposition maintenue et aucun contact pendant tout le déplacement. L'image finale répond à une question plus étroite que la trajectoire.

La collecte de correction fournit un deuxième enseignement. Le contrôleur de référence est interrogé sur des états plus proches de ceux du modèle, et le total des contacts baisse de 102 à 65 dans la comparaison fraîche. Cette amélioration reste insuffisante. Le modèle brut ne réussit aucun des dix-huit cas. On peut donc constater un effet utile des exemples supplémentaires sans conclure que le problème est résolu.

Le pilote multi-effectifs révèle une autre difficulté. Le réseau peut traiter les observations d'un plus grand groupe, mais le protocole complet ne réussit pas nécessairement. À 24 et 64 robots, la protection évite des contacts tout en laissant une préparation non résolue. Éviter les chocs ne suffit pas. Les robots doivent aussi s'accorder sur les places et se stabiliser.

La seconde architecture change le partage des responsabilités. Le mouvement nominal

d'attraction et de freinage est inscrit dans le calcul. Le réseau apprend surtout des corrections de voisinage. Cela semble mieux adapté aux situations testées. Les dix-huit cas réservés réussissent sans contact et sans filtre pendant le mouvement appris. Ce résultat appuie le choix pratique d'une commande structurée. Il ne démontre pas qu'un réseau libre aurait appris la même organisation avec le même budget.

La stabilité de la position finale est également liée à l'architecture. Le facteur d'activité annule les messages lorsque le robot est à sa cible et que le voisinage est immobile. Le réseau n'a donc pas à apprendre uniquement à partir des données qu'il doit cesser de corriger cet état. Ce comportement est prévu dans la conception du réseau, plutôt qu'appris spontanément à partir des exemples.

Enfin, les résultats de navigation répondent à une autre question. Les scènes finales sont franchies grâce à une carte connue, un détour commun et une commande protégée. Le planificateur tient compte de la largeur du groupe, ce qui corrige un défaut des premières scènes. On ne peut pas transférer ce succès au GAT structuré brut, qui n'a pas été évalué dans ces transports.

Comparaison avec les travaux antérieurs

Les travaux présentés dans l'état de l'art aident à expliquer nos choix. Leurs scores ne sont pas directement comparables aux nôtres. La formation de courbes par des règles, le déplacement collectif appris et la commande à partir de perceptions locales utilisent des tâches et des hypothèses différentes. Les écarts observés dans notre simulateur ne mesurent donc pas une supériorité sur ces approches.

Le rapprochement le plus utile porte sur la représentation locale. Les échanges permettent à un robot de tenir compte de ses voisins sans recevoir directement tout l'état physique de l'essaim. Notre implémentation rend cette limite visible dans ses entrées. Elle conserve toutefois une mission commune, une attribution préparée et, pour la navigation, une carte connue du planificateur.

L'apprentissage par imitation est un autre point commun. Dans notre cas, le contrôleur de référence de mouvement est local et ses commandes sont conservées dans les données. Le réseau n'a pas à reconstruire une décision fondée sur des états physiques éloignés qui lui seraient cachés. Cette cohérence facilite la lecture des exemples. Un décalage reste toutefois possible entre les états rencontrés par le contrôleur de référence et ceux visités par le réseau.

La différence entre perception simulée et perception réelle reste importante. Les caractéristiques relatives sont fournies exactement par le moteur. Un système utilisant une caméra ou un LiDAR devrait d'abord les estimer, ou apprendre une autre représentation. Une réussite dans notre modèle ne constitue donc pas une validation de cette étape de perception.

L'effet propre de l'attention reste à établir. Une comparaison entre le premier et le second réseau change trop d'éléments à la fois. Pour répondre à cette question, il faudrait retirer uniquement les messages ou les remplacer par une agrégation de référence, conserver les mêmes données et répéter l'entraînement. Nous n'utilisons pas les articles cités pour combler cette absence d'expérience.

Menaces à la validité

Validité interne

La validité interne concerne le lien entre les modifications et les effets observés. Les départs appariés réduisent le risque de comparer un contrôleur sur une scène facile à un autre sur une scène difficile. Les poids et les paramètres sont enregistrés, ce qui permet de vérifier quelle version a produit une trajectoire.

Ces précautions ne permettent toutefois pas d'isoler le rôle de chaque composant. La version structurée modifie à la fois la représentation, la commande nominale, les données et la fonction de coût. Son progrès ne peut pas être attribué uniquement à un terme. Le changement de planificateur et le rapprochement des obstacles produisent aussi des campagnes différentes ; leurs résultats restent séparés.

La sélection des poids s'appuie sur la validation. Cette partition n'est donc pas un ensemble totalement indépendant de la démarche de développement. Le test réservé est exécuté après la sélection, avec des graines annoncées. Si ses résultats servaient à modifier encore le modèle, une nouvelle réserve serait nécessaire pour évaluer la version suivante.

Les audits limitent certains risques techniques. Ils recalculent les mesures et vérifient l'intégration physique, les contacts et les interventions du filtre. Ils ne prouvent cependant pas que toutes les erreurs possibles du simulateur ont été éliminées. Une autre implémentation indépendante ou des essais matériels apporteraient un contrôle différent.

Validité externe

La réserve structurée comprend trois formes, deux départs par effectif et trois effectifs. Les dix-huit cas représentent seulement six configurations initiales distinctes. Les robots commencent près d'un anneau prévu. Il serait donc excessif d'annoncer un taux général de réussite à partir de ce seul ensemble.

La simulation ne comprend ni bruit de mesure, ni perte aléatoire de messages, ni différence de moteur. Les obstacles sont statiques et leur carte est déclarée. Les pannes de robot ne sont pas validées dans la version actuelle. Ce sont des limites du domaine étudié, pas des détails que le nombre d'exemples d'entraînement ferait disparaître.

Les effectifs 8 et 16, les formes non réservées et les dimensions adaptées dans l'interface

restent exploratoires selon le contrôleur. Pouvoir exécuter une scène ne signifie pas que ses résultats ont été validés. Il faut conserver cette distinction dans une présentation publique du travail.

Validité de construit

Cette validité concerne ce que les indicateurs représentent. L'erreur de couverture mesure des distances entre positions et places. Elle ne permet pas de savoir directement si un lecteur reconnaît la lettre. Nous n'avons pas conduit de questionnaire permettant d'affirmer qu'un symbole serait reconnu par une personne.

La MSE porte sur des accélérations et ne fournit pas un taux de classification. Une matrice de confusion n'est donc pas requise pour la tâche d'imitation actuelle. Elle aurait un autre rôle dans une étude distincte de reconnaissance des symboles. Le rappel et la précision ne doivent pas être ajoutés simplement pour donner davantage de chiffres.

Le temps de stabilisation comprend la préparation et l'attribution. À soixante-quatre robots, l'attente en nombre de tours domine la durée. Ce temps ne mesure ni le coût d'une passe du réseau ni la consommation d'énergie du groupe. Ces autres questions demanderaient leurs propres mesures.

Implications pour la conception de contrôleurs décentralisés

La première implication est de définir les rôles avant l'entraînement. Le réseau ne doit pas être chargé implicitement de résoudre une attribution qui n'apparaît ni dans ses entrées ni dans ses exemples. Dans notre système, la réservation est construite et vérifiée séparément. Il devient ainsi plus facile de comprendre les causes d'échec.

La deuxième implication est de choisir une structure de commande cohérente avec l'équilibre souhaité. L'attraction, le freinage et l'annulation des corrections à l'arrêt peuvent être prévus explicitement. Cette conception réduit la liberté du modèle, mais elle évite de lui demander de retrouver seul toutes les propriétés utiles à partir d'un échantillon limité.

La troisième est de conserver une comparaison brute et protégée. Une commande protégée peut être utile en pratique. Elle doit simplement être présentée comme une chaîne hybride et son filtre doit être mesuré. L'objectif d'un mouvement brut est une question différente, qui se juge sur les phases où la projection est réellement retirée.

La géométrie mérite la même attention que le réseau. Si les places sont trop proches ou hors de l'espace, un entraînement supplémentaire ne rend pas la consigne admissible. Vérifier la scène avant le lancement permet de distinguer une configuration impossible d'une erreur du modèle.

Enfin, les échanges doivent être décrits avec leur durée. La réservation locale fonctionne dans les cas testés, mais son budget fixe devient coûteux à grand effectif. Une condition d'arrêt plus précise pourrait améliorer la mission sans changer les poids du réseau. Elle demanderait cependant une nouvelle validation de l'accord entre robots.

Considérations pratiques et sociales

La simulation permet d'étudier le mouvement sans utiliser du matériel pendant l'apprentissage. Elle facilite la répétition d'un départ et l'analyse d'un contact. Son intérêt pratique tient aussi à cette possibilité de revoir une trajectoire lentement et de retrouver les paramètres qui l'ont produite.

Pour un utilisateur, les messages de l'interface doivent rester compréhensibles. Un avertissement sur un effectif exploratoire indique une limite de validation. Une cible bloquée indique une consigne impossible dans la scène. Une absence de stabilisation décrit un critère non atteint. Mélanger ces situations ferait perdre l'information nécessaire à une décision.

Le système n'a pas été conçu pour un usage opérationnel auprès de personnes. Aucun essai de sécurité matérielle n'a été réalisé. Les résultats ne justifient donc pas de supprimer les protections d'une plateforme réelle. Le passage au matériel doit être progressif et tenir compte des capteurs, des actionneurs et des conditions d'utilisation.

Sur le plan de la recherche, conserver les échecs et les paramètres permet au lecteur de juger la portée du travail. Les figures du mémoire sont reconstruites à partir du programme ou de ses archives. Elles montrent les cas indiqués dans leur légende ; elles ne remplacent pas une diversité d'expériences qui n'aurait pas été réalisée.

Suite expérimentale

La première suite consiste à élargir les départs sans modifier les poids après chaque difficulté. Un protocole devrait réserver de nouvelles graines et de nouvelles consignes, avec des critères fixés avant l'exécution. Il faudrait rapporter les échecs autant que les réussites et distinguer les graines de départ des graines d'entraînement.

Une deuxième suite doit mesurer l'utilité des messages. Une variante privée de voisins, une variante à moyenne simple et le modèle complet pourraient partager la même commande nominale et des budgets comparables. Plusieurs initialisations des poids aideraient à savoir si un écart est régulier ou dépend d'un entraînement particulier.

La troisième suite concerne le protocole d'attribution. Le budget en N^2 tours pourrait être remplacé en partie par une vérification d'accord permettant un arrêt plus tôt. Cette modification ne doit pas réduire silencieusement le temps en supposant l'accord acquis. Le cas d'une annonce retardée ou d'un graphe modifié devrait rester détectable.

La navigation pourrait ensuite être évaluée sur davantage de formes, d'effectifs et de cartes. Il faudrait séparer les cartes utilisées pour le développement de celles réservées au bilan. Les passages étroits, les destinations refusées et les détours impossibles doivent faire partie du compte rendu, même lorsqu'aucune trajectoire réussie n'est calculée.

Enfin, la perception et les communications peuvent être rendues moins idéales. Une étape raisonnable serait d'ajouter une perturbation à la fois, puis de mesurer la dégradation. Le passage à des robots physiques demanderait un modèle adapté au matériel et une estimation des caractéristiques locales. Ces pistes devront encore être mises en œuvre et évaluées.

Conclusion partielle

Les résultats montrent une réussite de formation dans le domaine contrôlé du modèle structuré et une réussite séparée de la navigation hybride. Ils montrent aussi que la MSE, l'absence de contact et l'achèvement de la mission ne sont pas des mesures interchangeables. Le faible nombre de départs, les observations idéales et les composants classiques limitent la généralisation. La conclusion générale rassemble maintenant ces apports et les perspectives qui en découlent.

Conclusion générale et perspectives

Ce mémoire étudie la formation d'une figure par des robots qui prennent leurs décisions à partir d'informations locales. La réponse a demandé plus qu'un réseau de neurones. Il a fallu définir une dynamique, construire les places, régler leur attribution, produire des exemples et mesurer les trajectoires complètes.

Le système actuel commande des accélérations dans un espace plan borné. Les données sont synthétiques. Un contrôleur de référence programmé fournit les observations et les commandes attendues. L'entraînement est une régression supervisée par imitation. Le catalogue propose 56 noms pour 54 géométries différentes, et le réseau reçoit sa propre cible ainsi que ses voisins directs. La géométrie et la réservation restent distinctes de la commande apprise.

Le premier modèle a permis de montrer les limites d'une lecture fondée sur la seule perte. Avec douze robots, les neuf cas réservés réussissent lorsqu'un filtre classique protège ses commandes. Sans ce filtre pendant le mouvement, les mêmes cas échouent et comptent 54 contacts malgré une bonne disposition finale. Une collecte complémentaire réduit ensuite les contacts sur une nouvelle campagne, sans obtenir un modèle brut fiable.

Le passage à une architecture structurée apporte un résultat plus favorable. Le mouvement nominal vers la cible et le freinage sont écrits explicitement ; les interactions voisines sont apprises. La version retenue réussit les dix-huit cas réservés à 12, 24 et 64 robots, sans contact et sans filtre pendant les phases de mouvement apprises. Le contrôleur de référence réussit également ces cas. Cette observation soutient le choix de conception dans les conditions évaluées.

La portée reste limitée. Les dix-huit cas utilisent trois formes et six configurations initiales distinctes, proches d'un anneau de préparation. La validation reste limitée aux départs et aux formes testés, avec des observations sans bruit. Les très petites erreurs finales sont celles d'une simulation idéale. Elles ne décrivent pas la précision de robots réels.

La navigation constitue une contribution complémentaire. Un planificateur sur carte connue prépare le trajet en tenant compte de toutes les places de la formation. Les scènes finales proches des cibles réussissent huit cas par contrôleur pour un carré à douze robots. Ce résultat appartient à une chaîne hybride avec préparation et protection classiques. Il n'établit pas un apprentissage autonome du contournement par le modèle brut.

Le travail apporte aussi une méthode de vérification. Les hypothèses, les partitions, les poids et les métriques sont reliés aux fichiers conservés. Les comparaisons appariées et les audits rendent les résultats vérifiables. La conservation des échecs permet de comprendre les changements apportés, sans transformer la dernière démonstration en garantie générale.

Les prochaines étapes sont d'élargir la réserve de départs, de répéter les entraînements

et d'isoler l'effet des messages. Le protocole d'attribution peut aussi être amélioré pour réduire l'attente à grand effectif. La navigation doit être examinée dans davantage de scènes avant d'envisager une perception moins idéale ou des robots physiques.

Ces résultats conduisent à une conclusion prudente. Une commande locale apprise peut former l'essaim sans contact dans le domaine testé, à condition de préciser la structure du modèle et la préparation commune. Les résultats les plus utiles sont ceux dont on connaît aussi les limites. C'est sur cette base que le système peut être prolongé.

Bibliographie

[1] Filipe de Avila Belbute-Peres et al. « End-to-end differentiable physics for learning and control ». In : *Advances in Neural Information Processing Systems*. T. 31. 2018.

[2] Manuele Brambilla et al. « Swarm robotics : a review from the swarm engineering perspective ». In : *Swarm Intelligence* 7 (2013), p. 1-41. doi : 10.1007/s11721-012-0075-2.

[3] Justin Gilmer et al. « Neural Message Passing for Quantum Chemistry ». In : *ICML*. 2017, p. 1263-1272.

[4] M. Ani Hsieh, Vijay Kumar et Luiz Chaimowicz. « Decentralized controllers for shape generation with robotic swarms ». In : *Robotica* 26.5 (2008), p. 691-701. doi : 10.1017/S0263574708004323.

[5] Chao Jiang, Xinchi Huang et Yi Guo. « End-to-end decentralized formation control using a graph neural network-based learning method ». In : *Frontiers in Robotics and AI* 10 (2023). doi : 10.3389/frobt.2023.1285412.

[6] Craig W. Reynolds. « Flocks, Herds, and Schools : A Distributed Behavioral Model ». In : *SIGGRAPH*. 1987, p. 25-34. doi : 10.1145/37401.37406.

[7] Stéphane Ross, Geoffrey Gordon et Drew Bagnell. « A reduction of imitation learning and structured prediction to no-regret online learning ». In : *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*. T. 15. Proceedings of Machine Learning Research. PMLR, 2011, p. 627-635.

[8] Franco Scarselli et al. « The graph neural network model ». In : *IEEE Transactions on Neural Networks* 20.1 (2009), p. 61-80. doi : 10.1109/TNN.2008.2005605.

[9] Ekaterina Tolstaya et al. « Learning Decentralized Controllers for Robot Swarms with Graph Neural Networks ». In : *CoRL*. 2020, p. 671-682.

[10] Petar Veličković et al. « Graph attention networks ». In : *International Conference on Learning Representations (ICLR)*. 2018.

[11] Manzil Zaheer et al. « Deep sets ». In : *Advances in Neural Information Processing Systems*. T. 30. 2017, p. 3391-3401.

Annexe A — Mode d'emploi

Pour utiliser l'application, il faut ouvrir un terminal dans le dossier du projet, puis démarrer le serveur. Les commandes suivantes permettent de lancer la démonstration avec les modèles déjà entraînés.

Lancement de l'application

Sous Windows, le script PowerShell permet de démarrer le serveur avec un environnement Python disponible sur l'ordinateur.

```
.\demo\start-reference.ps1
```

On peut aussi installer les bibliothèques nécessaires et lancer le serveur directement avec Python.

```
python -m pip install -r requirements.txt
python -m demo.reference_server
```

L'application s'ouvre dans le navigateur à l'adresse `http://127.0.0.1:8765/`. Le terminal doit rester ouvert pendant la démonstration. Pour arrêter le serveur, il suffit d'appuyer sur Ctrl+C dans ce terminal. Le lancement utilise les modèles enregistrés sans refaire l'entraînement.

Déroulement d'un essai

Avant de lancer un essai, l'utilisateur choisit l'expérience, la forme, le nombre de robots et le contrôleur. Le numéro de départ permet de retrouver la même disposition initiale lorsque les autres réglages restent identiques.

La trajectoire est calculée avant le début de l'animation. Les robots passent d'abord par un cercle de rendez-vous, puis se répartissent les places et construisent la forme demandée. Le bouton Résultat final affiche directement le dernier état de la simulation.

Les commandes de lecture permettent de mettre l'animation en pause, de l'accélérer ou de revoir un instant précis. L'affichage des liens entre voisins, des trajectoires et des places aide à comprendre les mouvements. Les métriques présentées sous l'animation portent sur l'essai complet.

Annexe B — Les expériences

L'application propose sept expériences. Les choix disponibles dépendent du nombre de robots et du contrôleur utilisé. Les résultats présentés dans le mémoire concernent les configurations effectivement évaluées.

Formation et changement de forme

Former l'essaim. Les robots construisent une lettre, un nombre ou une figure du catalogue après la préparation et l'attribution des places. L'erreur finale, les vitesses et les contacts permettent de juger le résultat. Le test final du GAT structuré utilise le rectangle, la lettre C et le nombre 3, avec 12, 24 et 64 robots.

Changer de formation. La deuxième étape reprend les positions et les vitesses obtenues à la fin de la première. Les places sont réattribuées pour la nouvelle forme. L'évaluation tient compte de chaque étape et des contacts sur toute la séquence. Cette expérience n'est pas proposée au modèle structuré ni à 64 robots dans la version étudiée.

Déplacement collectif

Déplacer en ligne droite. Les positions cibles se déplacent ensemble le long d'un trajet sans obstacle. La forme doit être maintenue et le groupe doit atteindre la destination.

Contourner un obstacle. Le groupe suit un parcours qui passe à côté de l'obstacle. Le planificateur vérifie le passage de l'ensemble des places et adapte le détour si nécessaire.

Obstacle sur la route. Le trajet demandé est direct, mais un obstacle gêne certaines places. Le planificateur classique cherche un détour commun compatible avec la largeur de la formation.

Naviguer avec plusieurs obstacles. La scène comporte de cinq à dix obstacles proches des cibles. La campagne décrite dans le mémoire compare le contrôleur classique et le GAT initial protégé pour un carré de douze robots. Elle n'évalue pas le GAT structuré brut en navigation.

Vérifier une destination obstruée. Si une place finale est physiquement bloquée, la demande est signalée comme impossible. Ce signalement évite de lancer une mission dont la destination est inaccessible. Il reste distinct d'une formation réussie.

Annexe C — Résultats des simulations

Les fichiers enregistrés permettent de retrouver les résultats présentés dans le mémoire et d'examiner les trajectoires. Les tests du programme et l'évaluation du modèle sont deux vérifications différentes.

Vérification du programme

Les tests du logiciel s'exécutent depuis la racine du projet.

```
python -m pytest tests -q -p no:cacheprovider
```

La commande suivante produit un rapport à partir des résultats déjà enregistrés. Un nouveau dossier de sortie permet de conserver les rapports précédents.

```
python -m demo.comparison_report --output outputs/lecture
```

Fichiers des expériences

Le rapport `008d-local-target-expanded-test.json` documente les neuf essais du GAT initial. Le rapport `014-vector-fresh-test.json` décrit les dix-huit essais réservés du modèle structuré. Le rapport `019-near-targets-evaluation.json` présente la campagne finale près des obstacles. Ces fichiers se trouvent dans `tasks/evidence/`. Les audits associés vérifient les trajectoires enregistrées.

Les journaux `run.json` des deux entraînements se trouvent dans leurs dossiers respectifs sous `outputs/imitation/`. Les poids conservés sont les fichiers `best.pt`. Le manifeste `latex/generated/sources.json` rassemble les empreintes des données utilisées pour les figures et les tableaux du mémoire.

Interprétation des résultats

Pour comparer deux contrôleurs, on conserve la même forme, le même départ, le même effectif, les mêmes dimensions et les mêmes seuils. L'export contient les positions, les vitesses et les métriques. Une faible erreur finale ne suffit pas si un contact est survenu.

La stabilisation exige une erreur de formation au plus égale à 0,10 m et des vitesses au plus égales à 0,05 m/s pendant une seconde. La réussite exige aussi l'absence de contact sur l'essai. En navigation s'ajoutent l'arrivée à moins de 0,10 m et une déformation en route au plus égale à 0,20 m. Le temps indiqué est un temps simulé.